



دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

مستند جامع و کاربردی مفاهیم T-SQL

T-SQL

تهیه و تدوین

مهران تقوی افخم

مهندسی کامپیوتر

دانشگاه صنعتی شریف

mehran.taghavi75@sharif.edu



@Taghavi_Mehran



۱ مقدمه‌ای بر Transact-SQL (T-SQL)

این فصل خلاصه‌ای ساده شده و به زبان فارسی از اولین مازول مسیر یادگیری مایکروسافت Query and modify data with Transact-SQL است؛ مازول Introduction to Transact-SQL که پایه و شروع کل مسیر یادگیری T-SQL محسوب می‌شود.

۱.۱ SQL چیست و T-SQL چه تفاوتی با آن دارد؟

SQL مخفف Structured Query Language است؛ زبانی که برای ارتباط با دیتابیس‌های رابطه‌ای استفاده می‌شود. با SQL می‌توان داده را از دیتابیس خواند یا آن را تغییر داد. برای مثال، دستور معروف SELECT برای واکشی مجموعه‌ای از ردیف‌های داده به کار می‌رود. سیستم‌های مدیریت دیتابیس رابطه‌ای معروفی مثل MariaDB, PostgreSQL, MySQL, SQL Server و Oracle همگی از SQL استفاده می‌کنند. استاندارد به نام ANSI SQL توسط مؤسسه‌ی استاندارد آمریکا (ANSI) تعریف شده، اما هر شرکتی (وندور) روی این استاندارد پایه، ویژگی‌های اختصاصی خودش را هم اضافه می‌کند. Transact-SQL یا T-SQL همان گویش (dialect) مخصوص محصولات مایکروسافت است؛ یعنی SQL Server, Microsoft Fabric, Azure SQL Database و مشابه آن‌ها. T-SQL علاوه بر دستورات پایه‌ی SQL (مثل SELECT INSERT UPDATE DELETE که در همه‌ی دیتابیس‌ها مشترک هستند)، امکانات اضافه‌ای هم دارد؛ مثلاً نوشتن stored procedure و function (کدهایی که داخل خود دیتابیس ذخیره و اجرا می‌شوند) و همچنین مدیریت حساب‌های کاربری.

۲.۱ SQL یک زبان اعلانی (Declarative) است

زبان‌های برنامه‌نویسی را می‌توان به دو دسته تقسیم کرد:

- **رویه‌ای (Procedural):** شما دقیقاً مرحله به مرحله می‌گویید کامپیوتر چه کاری انجام دهد.
 - **اعلانی (Declarative):** شما فقط می‌گویید چه نتیجه‌ای می‌خواهید؛ جزئیات نحوه‌ی رسیدن به آن نتیجه بر عهده‌ی موتور اجراکننده است.
- SQL هرچند مقداری قابلیت رویه‌ای هم دارد، اما در حالت معمول (وقتی داده کوئری می‌گیرید) یک زبان اعلانی است. شما توصیف می‌کنید چه داده‌ای می‌خواهید، و Processor Query (پردازشگر کوئری در SQL Server) با استفاده از آمار داده‌ها و ایندکس‌های موجود، بهترین نقشه‌ی اجرا (Query Plan) را خودش تصمیم می‌گیرد.

۳.۱ داده‌ی رابطه‌ای (Relational Data)

در یک دیتابیس رابطه‌ای، داده در چندین جدول (که از نظر فنی به آن‌ها relation هم گفته می‌شود) سازمان‌دهی می‌شود. هر جدول نماینده‌ی یک نوع موجودیت (entity) است، مثل مشتری، محصول یا سفارش.

- ویژگی‌های هر موجودیت (مثلاً نام مشتری، قیمت محصول، تاریخ سفارش) همان ستون‌های جدول هستند.
 - هر ردیف جدول، یک نمونه‌ی مشخص از آن موجودیت است (مثلاً یک مشتری خاص).
- جدول‌ها با استفاده از ستون‌های کلید (Key) به هم مرتبط می‌شوند:
- هر جدول یک کلید اصلی (Primary Key) دارد که هر ردیف را به طور یکتا شناسایی می‌کند.
 - جدول‌های مرتبط، این کلید را به عنوان کلید خارجی (Foreign Key) در خودشان نگه می‌دارند تا رابطه برقرار شود.

۱.۳.۱ مثال

فرض کنید یک دیتابیس فروش شامل ۴ جدول داریم:

Customer (مشتری) — کلید اصلی: CustomerID

• **SalesOrderHeader** (سرپرگ سفارش) — کلید اصلی: OrderID، و کلید خارجی CustomerID که به جدول Customer اشاره می‌کند

• **SalesOrderDetail** (جزئیات سفارش) — کلید اصلی ترکیبی از OrderID و LineItemNo؛ همچنین OrderID و ProductID به عنوان کلید خارجی نگه‌داری می‌شوند

• **Product** (محصول) — کلید اصلی: ProductID

این ساختار همان چیزی است که در فصل JOIN استفاده می‌شود تا داده‌ی این جدول‌ها را با هم ترکیب کنیم.

۴.۱ پردازش مبتنی بر مجموعه (Set-Based Processing)

یکی از مهم‌ترین پایه‌های نظری دیتابیس‌های رابطه‌ای، **نظریه‌ی مجموعه‌ها (Set Theory)** است. به زبان ساده، یک **مجموعه** یعنی گردهمایی از اشیاء مشخص و متمایز که به عنوان یک کل در نظر گرفته می‌شوند.

• جدول Customer را می‌توان مجموعه‌ی «تمام مشتریان» در نظر گرفت.

• نتیجه‌ی یک دستور SELECT هم خودش یک مجموعه است.

نکته‌ی خیلی مهم: وقتی با T-SQL کار می‌کنید، باید همیشه به کل مجموعه فکر کنید، نه یک ردیف در یک زمان. این طرز فکر همان چیزی است که به آن **کدنویسی مبتنی بر مجموعه (Set-Based)** می‌گویند، در مقابل پردازش ردیف به ردیف.

نکته‌ی دیگر: در نظریه‌ی مجموعه‌ها هیچ ترتیب مشخصی برای اعضا وجود ندارد. در جدول‌های دیتابیس هم همین‌طور است — مفهومی به نام «ردیف اول» یا «ردیف آخر» وجود ندارد. اگر می‌خواهید نتیجه به ترتیب خاصی برگردد، باید آن را صریحاً با ORDER BY مشخص کنید.

۵.۱ کار با اسکیمای (Schema)

در SQL، Server، جدول‌ها داخل اسکیمای (Schema) تعریف می‌شوند. اسکیمای یک فضای نام منطقی (namespace) در دیتابیس ایجاد می‌کند.
برای مثال:

• جدول Customer می‌تواند داخل اسکیمای Sales باشد.

• جدول Product می‌تواند داخل اسکیمای Production باشد.

• حتی ممکن است یک جدول Order هم در اسکیمای Sales باشد (سفارش‌های مشتری) و هم در اسکیمای Production (سفارش از تأمین‌کنندگان)؛ اسکیمای به ما کمک می‌کند این دورا از هم جدا نگه داریم.

۱.۵.۱ نام‌گذاری کامل (Fully Qualified Name)

SQL Server از یک سیستم نام‌گذاری چندسطحی استفاده می‌کند تا جدول‌های هم‌نام در اسکیمای مختلف با هم اشتباه گرفته نشوند. نام کامل یک آبجکت شامل این بخش‌هاست:

1 **Table . Schema . Database . Server**

مثال: Server1.StoreDB.Sales.Order

در عمل، وقتی داخل یک دیتابیس مشخص کار می‌کنید، معمولاً کافی است فقط اسکیمای نام جدول را بنویسید؛ مثلاً Sales.Order.

۶.۱ انواع دستورات SQL

دستورات زبان SQL در چند دسته‌ی اصلی قرار می‌گیرند:

|p۰.۱۰|p۰.۲۲|p۰.۲۶|p۰.۳۰|

دسته نام کامل دستورات نمونه کاربرد

DML Data Manipulation Language SELECT, INSERT, UPDATE, DELETE کوئری گرفتن و تغییر داده
DDL Data Definition Language CREATE, ALTER, DROP تعریف و مدیریت چرخه‌ی عمر آبجکت‌هایی مثل جدول،

ویو و پروسجر

DCL Data Control Language GRANT, REVOKE, DENY مدیریت مجوزهای امنیتی کاربران و آبجکت‌ها

نکته‌ای که گاهی دیده می‌شود: بعضی منابع از TCL (Transaction Control Language) هم به عنوان دسته‌ی جداگانه یاد می‌کنند. همچنین در برخی تقسیم‌بندی‌ها، DML فقط به دستورات تغییردهنده (INSERT/UPDATE/DELETE) گفته می‌شود و SELECT را جزو دسته‌ی جدایی به نام DQL (Data Query Language) قرار می‌دهند. در این مسیر یادگیری، تمرکز اصلی روی دستورات DML است؛ چون این دستورات پرکاربردترین ابزار تحلیل‌گران داده برای گزارش‌گیری و همچنین ابزار اصلی توسعه‌دهندگان برای عملیات CRUD (ایجاد، خواندن، به‌روزرسانی، حذف داده) هستند.

۷.۱ آشنایی با دستور SELECT

دستور SELECT مهم‌ترین و پرکاربردترین دستور T-SQL است و بیشترین تنوع و گزینه را در بین همه‌ی دستورات DML دارد.

۱.۷.۱ ترتیب نوشتن در مقابل ترتیب اجرا

نکته‌ی کلیدی: ترتیبی که یک کوئری SELECT را می‌نویسید، همان ترتیبی نیست که Server SQL آن را پردازش می‌کند. به این کوئری نمونه توجه کنید:

```
1 SELECT OrderDate, COUNT(OrderID) AS Orders
2 FROM Sales.SalesOrder
3 WHERE Status = 'Shipped'
4 GROUP BY OrderDate
5 HAVING COUNT(OrderID) > 1
6 ORDER BY OrderDate DESC;
```

اما Server SQL این کوئری را به این ترتیب منطقی پردازش می‌کند:

۱. **FROM** — اول اجرا می‌شود؛ منبع ردیف‌ها را مشخص می‌کند و یک جدول مجازی می‌سازد.
۲. **WHERE** — روی جدول مجازی، ردیف‌هایی که با شرط مطابقت ندارند فیلتر می‌شوند.
۳. **GROUP BY** — ردیف‌های باقی‌مانده بر اساس مقادیر مشترک، گروه‌بندی می‌شوند. از این مرحله به بعد فقط می‌توان به ستون‌های GROUP BY یا توابع تجمعی (aggregate) دسترسی داشت.
۴. **HAVING** — کل گروه‌ها (نه تک‌تک ردیف‌ها) بر اساس یک شرط فیلتر می‌شوند.
۵. **SELECT** — در نهایت مشخص می‌شود کدام ستون‌ها در خروجی نمایش داده شوند. چون SELECT بعد از بقیه پردازش می‌شود، نام مستعار (alias) ساخته شده در SELECT را نمی‌توان در GROUP BY یا HAVING استفاده کرد.
۶. **BY ORDER** — آخرین مرحله؛ نتیجه را مرتب می‌کند.

تنها بخش الزامی هر SELECT خود عبارت SELECT است (که به تنهایی هم قابل استفاده است). معمولاً FROM هم همراهش می‌آید تا مشخص شود از کدام جدول کوئری می‌گیریم.

۲.۷.۱ انتخاب همه‌ی ستون‌ها با *

```
1 SELECT * FROM Production.Product;
```

استفاده از * (که به آن **star** هم گفته می‌شود) برای تست سریع مناسب است، اما در کدهای واقعی و production توصیه نمی‌شود، چون:

- اگر بعداً ستونی به جدول اضافه یا جابه‌جا شود، خروجی کوئری بدون اطلاع شما تغییر می‌کند و می‌تواند اپلیکیشن یا گزارش را دچار مشکل کند.
- واکنشی ستون‌های غیرضروری، به خصوص روی جدول‌های بزرگ، باعث افت کارایی می‌شود.

۳.۷.۱ انتخاب ستون‌های مشخص

```
1 SELECT ProductID, Name, ListPrice, StandardCost
2 FROM Production.Product;
```

با نوشتن اسم ستون‌ها، دقیقاً کنترل دارید چه ستون‌هایی و با چه ترتیبی برگردانده شوند.

۴.۷.۱ نوشتن عبارت (Expression) در SELECT

می‌توان به جای برگرداندن مستقیم یک ستون، از عملگرها برای ترکیب یا محاسبه استفاده کرد؛ نتیجه باید یک مقدار تکی (scalar) باشد:

```
1 SELECT ProductID,  
2      Name + ' (' + ProductNumber + ')',  
3      ListPrice - StandardCost  
4 FROM Production.Product;
```

در این مثال، عملگر + برای الحاق دو رشته‌ی متنی و عملگر - برای تفریق دو مقدار عددی استفاده شده است. توجه به نوع داده (Data Type) ستون‌ها در این‌گونه عبارات ضروری است.

۵.۷.۱ نام‌گذاری ستون خروجی با Alias

وقتی از عبارت استفاده می‌کنید، ستون خروجی معمولاً نام مشخصی ندارد. با کلمه‌ی AS می‌توانید برای آن یک نام (alias) بگذارید:

```
1 SELECT ProductID AS ID,  
2      Name + ' (' + ProductNumber + ') ' AS ProductName,  
3      ListPrice - StandardCost AS Markup  
4 FROM Production.Product;
```

نوشتن AS اختیاری است، اما برای خوانایی بهتر توصیه می‌شود همیشه نوشته شود.

۶.۷.۱ نکات فرمت‌بندی کد

برای اینکه کد T-SQL خوانا تر و قابل نگهداری تر باشد:

- کلمات کلیدی مثل SELECT, FROM, AS را با حروف بزرگ بنویسید.
- هر عبارت اصلی کوئری (SELECT FROM WHERE و...) را در یک خط جدید شروع کنید.
- اگر لیست SELECT چند ستون یا عبارت دارد، هر کدام را در یک خط جداگانه بنویسید.
- خط‌هایی که زیرمجموعه یا ستون‌های یک بخش هستند را تورفتگی (indent) بدهید.

۸.۱ انواع داده (Data Types) و تبدیل آن‌ها

هر ستون یا متغیر در T-SQL یک نوع داده (Data Type) مشخص دارد که رفتار آن در عبارات مختلف را تعیین می‌کند. مهم‌ترین دسته‌های نوع داده در Server SQL عبارت‌اند از:

- عددی دقیق: tinyint, smallint, int, bigint, decimal/numeric, bit, money, smallmoney
- عددی تقریبی: float, real
- متنی: char, varchar, nchar, nvarchar, text, ntext
- تاریخ/زمان: date, time, datetime, datetime2, smalldatetime, datetimeoffset
- باینری: binary, varbinary, image
- سایر: cursor, hierarchyid, sql_variant, table, timestamp, uniqueidentifier, xml, ge-ography, geometry

۱.۸.۱ تبدیل نوع داده

مقادیر با نوع داده‌ی سازگار به صورت خودکار (implicit) تبدیل می‌شوند؛ مثلاً جمع یک عدد صحیح با یک عدد اعشاری. اما در بعضی موارد باید تبدیل را صریح (explicit) انجام دهید؛ مثلاً الحاق مستقیم یک varchar با یک decimal معمولاً خطا می‌دهد، مگر ابتدا مقدار عددی را به رشته تبدیل کنید.

T-SQL چند تابع برای تبدیل صریح نوع داده در اختیاران می‌گذارد:

TRY_CAST و CAST

تابع استاندارد ANSI برای تبدیل نوع داده. اگر مقدار با نوع مقصد سازگار نباشد، CAST خطا می‌دهد؛ اما TRY_CAST در چنین حالتی به جای خطا، مقدار NULL برمی‌گرداند:

```
1 SELECT CAST(ProductID AS varchar(4)) + ': ' + Name AS ProductName
2 FROM Production.Product;
3
4 -- TRY_CAST NULL
5 SELECT TRY_CAST(Size AS integer) AS NumericSize
6 FROM Production.Product;
```

TRY_CONVERT و CONVERT

مشابه CAST است، اما مخصوص T-SQL بوده و یک پارامتر اضافه هم دارد که فرمت خروجی (مخصوصاً برای تاریخ) را کنترل می‌کند:

```
1 SELECT CONVERT(varchar(4), ProductID) + ': ' + Name AS ProductName
2 FROM Production.Product;
3
4 SELECT SellStartDate,
5        CONVERT(varchar(10), SellStartDate, 101) AS FormattedStartDate
6 FROM SalesLT.Product;
```

TRY_PARSE و PARSE

برای تبدیل رشته‌های فرمت‌دار (مثل تاریخ یا مبلغ) به نوع عددی یا تاریخی مناسب:

```
1 SELECT PARSE('01/01/2021' AS date) AS DateValue,
2        PARSE('$199.99' AS money) AS MoneyValue;
```

STR

یک مقدار عددی را به varchar تبدیل می‌کند:

```
1 SELECT ProductID, '$' + STR(ListPrice) AS Price
2 FROM Production.Product;
```

۹.۱ کار با مقدارهای NULL

مقدار NULL یعنی «بدون مقدار» یا «نامشخص». این با صفر، رشته‌ی خالی یا فاصله‌ی خالی فرق دارد — آن مقادیر مشخص هستند، اما NULL یعنی مقدار اصلاً معلوم نیست (مثلاً وقتی مشتری هنوز ایمیلش را وارد نکرده).

NULL هیچ چیز نیست: نه با چیزی برابر است، نه نابرابر؛ نه از چیزی بزرگ‌تر است، نه کوچک‌تر. به همین دلیل T-SQL توابع مخصوصی برای مدیریت NULL دارد:

ISNULL ۱.۹.۱

دو آرگومان می‌گیرد. اگر آرگومان اول NULL باشد، آرگومان دوم را برمی‌گرداند؛ در غیر این صورت خود آرگومان اول را برمی‌گرداند:

```
1 SELECT FirstName,
2        ISNULL(MiddleName, 'None') AS MiddleIfAny,
3        LastName
4 FROM Sales.Customer;
```

نکته: مقدار جایگزین باید هم‌نوع با ستون اصلی باشد و بهتر است مقداری انتخاب شود که در داده‌های واقعی تکرار نشود.

۲.۹.۱ COALESCE

ISNULL استاندارد ANSI نیست، اما COALESCE هست. تفاوت اصلی این است که COALESCE می‌تواند چند آرگومان بگیرد و اولین مقدار غیر NULL را برمی‌گرداند. اگر همه‌ی آرگومان‌ها NULL باشند، نتیجه هم NULL خواهد بود.

```
1 SELECT COALESCE(expression1, expression2, ...)
```

مثال: فرض کنید یک جدول حقوق دارید که هر کارمند فقط یکی از سه فیلد (نرخ ساعتی، حقوق هفتگی، پورسانت) را پر کرده و بقیه NULL هستند:

```
1 SELECT EmployeeID,
2         COALESCE(HourlyRate * 40,
3                   WeeklySalary,
4                   Commission * SalesQty) AS WeeklyEarnings
5 FROM HR.Wages;
```

۳.۹.۱ NULLIF

دو آرگومان می‌گیرد؛ اگر برابر باشند NULL برمی‌گرداند، در غیر این صورت آرگومان اول را برمی‌گرداند. این تابع برای پاک‌سازی داده کاربرد دارد؛ مثلاً وقتی می‌خواهید مقدار صفر را به جای «بدون تخفیف»، به صورت NULL نمایش دهید:

```
1 SELECT SalesOrderID,
2         ProductID,
3         UnitPrice,
4         NULLIF(UnitPriceDiscount, 0) AS Discount
5 FROM Sales.SalesOrderDetail;
```

۱۰.۱ جمع‌بندی و نکات طلایی

- SQL زبان کوئری گرفتن از دیتابیس‌های رابطه‌ای است؛ T-SQL گویش مخصوص مایکروسافت از آن است.
- SQL یک زبان اعلانی است: شما نتیجه‌ی موردنظرتان را توصیف می‌کنید، نه مراحل رسیدن به آن را.
- داده در دیتابیس رابطه‌ای در قالب جدول‌ها با کلید اصلی و خارجی به هم مرتبط می‌شود.
- همیشه به داده به شکل مجموعه فکر کنید، نه ردیف به ردیف؛ و برای ترتیب دلخواه حتماً ORDER BY بنویسید.
- جدول‌ها داخل اسکیمای سازمان‌دهی می‌شوند (مثل Sales.Customer).
- دستورات SQL به سه دسته‌ی اصلی تقسیم می‌شوند: DML (کوئری/تغییر داده)، DDL (تعریف آبجکت‌ها)، DCL (مجوزها).
- ترتیب نوشتن یک SELECT با ترتیب اجرای منطقی آن فرق دارد: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.
- از * SELECT در کدهای واقعی خودداری کنید؛ ستون‌های مشخص را نام ببرید.
- برای تبدیل نوع داده از TRY_CAST/CAST، TRY_CONVERT/CONVERT یا TRY_PARSE/PARSE استفاده کنید.
- NULL یعنی «نامشخص»، نه صفر یا خالی. برای مدیریت آن از COALESCE، ISNULL و NULLIF کمک بگیرید.

منبع

این نوشته بر اساس مطالب مازول رسمی مایکروسافت لرن تهیه شده است: [Learn Microsoft — Transact-SQL to Introduction](#)

۲ مرتب سازی و فیلتر کردن نتایج در T-SQL

این فصل خلاصه‌ای ساده شده و به زبان فارسی از مازول دوم مسیر یادگیری مایکروسافت Query and modify data with Transact-SQL است؛ مازول Sort and filter results in T-SQL که بعد از مازول مقدماتی (Introductory to Transact-SQL) و قبل از مبحث JOIN قرار می‌گیرد.

۱.۲ مقدمه

وقتی داده‌ای از دیتابیس واکنشی می‌کنیم، معمولاً دو نیاز رایج داریم:

- نتایج را به یک ترتیب خاص مرتب کنیم.
 - فقط ردیف‌هایی را ببینیم که شرایط مشخصی دارند (فیلتر کردن).
- در T-SQL، مرتب سازی با عبارت ORDER BY و فیلتر کردن با عبارت WHERE انجام می‌شود. در این مازول یاد می‌گیریم:

- چطور نتایج را با ORDER BY مرتب کنیم.
- چطور با TOP فقط تعداد مشخصی از ردیف‌های بالای نتیجه‌ی مرتب شده را نگه داریم.
- چطور با OFFSET-FETCH نتایج را صفحه بندی کنیم.
- چطور با DISTINCT ردیف‌های تکراری را حذف کنیم.
- چطور با WHERE و انواع عملگرهای مقایسه‌ای، داده را فیلتر کنیم.

۲.۲ مرتب سازی نتایج با ORDER BY

یکی از نکاتی که در مازول اول یاد گرفتیم این بود: دیتابیس رابطه‌ای هیچ ترتیب پیش فرضی برای ردیف‌های یک جدول ندارد. Server SQL هیچ تضمینی برای ترتیب فیزیکی ردیف‌ها نمی‌دهد و تنها راه کنترل ترتیب خروجی، استفاده از عبارت ORDER BY است. در ترتیب پردازش منطقی یک SELECT، عبارت ORDER BY آخرین مرحله است؛ یعنی بعد از اینکه همه‌ی مراحل دیگر (FROM WHERE GROUP BY HAVING SELECT) تمام شدند، در نهایت نتیجه مرتب می‌شود.

۱.۲.۲ نحو کلی

```
1 SELECT <column list>
2 FROM <table source>
3 ORDER BY <order by list> [ASC|DESC];
```

در لیست ORDER BY می‌توانید موارد زیر را بنویسید:

- نام ستون‌ها: نتیجه ابتدا بر اساس ستون اول مرتب می‌شود، سپس در داخل هر گروه، بر اساس ستون دوم و همین‌طور به ترتیب ادامه پیدا می‌کند.
- نام مستعار (Alias) ستون‌ها: چون ORDER BY بعد از SELECT پردازش می‌شود، به alias‌های تعریف شده در SELECT هم دسترسی دارد.
- شماره‌ی موقعیت ستون در لیست SELECT: مثلاً 2 ORDER BY. این روش توصیه نمی‌شود چون خوانایی کد را کم می‌کند و نگهداری آن سخت‌تر است؛ فقط گاهی برای دیباگ عبارت‌های پیچیده مفید است.
- ستون‌هایی که در SELECT نیامده‌اند، ولی در جدول‌های FROM موجودند. البته اگر از DISTINCT استفاده کرده باشید، هر ستونی که در ORDER BY می‌آید حتماً در لیست SELECT هم باشد.

۲.۲.۲ جهت مرتب‌سازی

• `ASC` = صعودی (پیش فرض: از A تا Z، از ۰ تا ۹)

• `DESC` = نزولی (از Z تا A، از ۹ تا ۰)

هرستون می‌تواند جهت مرتب‌سازی مستقل خودش را داشته باشد:

```
1 SELECT ProductCategoryID AS Category, ProductName, ListPrice
2 FROM Production.Product
3 ORDER BY Category ASC, ListPrice DESC;
```

در این مثال، ابتدا بر اساس دسته‌بندی به صورت صعودی و در داخل هر دسته، بر اساس قیمت به صورت نزولی مرتب می‌شود.

۳.۲ محدود کردن نتایج با TOP

TOP یک ویژگی اختصاصی مایکروسافت (نه استاندارد ANSI) است که به شما اجازه می‌دهد فقط تعداد مشخصی از ردیف‌های بالای نتیجه را بگیرید؛ چه به صورت عدد ثابت و چه به صورت درصد.

۱.۳.۲ نحو پایه

```
1 SELECT TOP (N) <column list>
2 FROM <table source>
3 WHERE <search condition>
4 ORDER BY <order by list> [ASC|DESC];
```

مثال: گرفتن ۱۰ محصول گران قیمت:

```
1 SELECT TOP 10 Name, ListPrice
2 FROM Production.Product
3 ORDER BY ListPrice DESC;
```

نکته‌ی خیلی مهم: TOP به `ORDER BY` وابسته است تا معنای درستی داشته باشد. اگر بدون `ORDER BY` استفاده شود، هیچ تضمینی نیست کدام ردیف‌ها برگردانده می‌شوند — هر ۱۰ ردیف دلخواهی ممکن است انتخاب شود.

۲.۳.۲ گزینه‌ی TIES WITH

اگر بخواهید علاوه بر N ردیف اول، ردیف‌هایی را هم که مقدارشان با آخرین ردیف انتخاب شده برابر است شامل کنید، از `WITH TIES` استفاده کنید:

```
1 SELECT TOP 10 WITH TIES Name, ListPrice
2 FROM Production.Product
3 ORDER BY ListPrice DESC;
```

با این گزینه، ممکن است بیشتر از ۱۰ ردیف برگردد؛ چون همه‌ی محصولات که قیمتشان دقیقاً برابر با نفر دهم است هم اضافه می‌شوند. استفاده از `WITH TIES` بستگی به شناخت شما از داده و نیاز کوئری دارد.

۳.۳.۲ گزینه‌ی PERCENT

به جای عدد ثابت، می‌توانید درصدی از کل ردیف‌های واجد شرایط را بگیرید:

```
1 SELECT TOP 10 PERCENT Name, ListPrice
2 FROM Production.Product
3 ORDER BY ListPrice DESC;
```

`PERCENT` را می‌توان با `WITH TIES` هم ترکیب کرد. توجه کنید که برای محاسبه‌ی تعداد ردیف، مقدار درصد همیشه به بالا گرد می‌شود.

۴.۳.۲ نکات مهم TOP

- TOP مخصوص T-SQL است و در همه‌ی دیتابیس‌ها وجود ندارد.
- به‌تنهایی امکان «رد کردن (skip)» چند ردیف اول را ندارد.
- چون TOP به ORDER BY وابسته است، نمی‌توانید یک ترتیب برای انتخاب ردیف‌ها و ترتیب دیگری برای نمایش خروجی داشته باشید.

۴.۲ صفحه‌بندی نتایج با OFFSET-FETCH

OFFSET-FETCH یک افزونه روی ORDER BY است که به شما اجازه می‌دهد فقط بازه‌ای از ردیف‌های نتیجه را برگردانید. این ویژگی برای پیاده‌سازی صفحه‌بندی (Paging) بسیار کاربردی است؛ مثلاً وقتی می‌خواهید نتایج را ۱۰ تا ۱۰ تا در صفحات مختلف نمایش دهید. نکته‌ی مهم: هر کوئری که شامل OFFSET-FETCH است، مستقل از کوئری‌های قبلی اجرا می‌شود. Server SQL هیچ وضعیتی (state) را بین درخواست‌ها نگه نمی‌دارد؛ ردیابی اینکه کاربر الان در کدام صفحه است، بر عهده‌ی کد سمت کلاینت (اپلیکیشن) است.

۱.۴.۲ نحو

```
1 OFFSET { constant_or_expr } { ROW | ROWS }
2 [FETCH { FIRST | NEXT } { constant_or_expr } { ROW | ROWS } ONLY]
```

۲.۴.۲ مثال

گرفتن ۱۰ ردیف اول:

```
1 SELECT ProductID, ProductName, ListPrice
2 FROM Production.Product
3 ORDER BY ListPrice DESC
4 OFFSET 0 ROWS -- no rows skipped
5 FETCH NEXT 10 ROWS ONLY; -- get the next 10 rows
```

گرفتن ۱۰ ردیف بعدی (یعنی صفحه‌ی دوم):

```
1 SELECT ProductID, ProductName, ListPrice
2 FROM Production.Product
3 ORDER BY ListPrice DESC
4 OFFSET 10 ROWS -- skip the first 10 rows
5 FETCH NEXT 10 ROWS ONLY; -- get the next 10 rows
```

۳.۴.۲ نکات مهم OFFSET-FETCH

- بخش OFFSET الزامی است، اما FETCH اختیاری است؛ اگر FETCH نوشته نشود، همه‌ی ردیف‌های بعد از OFFSET برگردانده می‌شوند.
- کلمات ROWS/ROW با هم قابل تعویض‌اند، و همین‌طور NEXT/FIRST هم قابل تعویض‌اند — این کار فقط برای طبیعی‌تر شدن نحو زبان است.
- برای اینکه هنگام جابه‌جایی بین صفحات نتیجه‌ی دقیق و قابل‌اعتمادی داشته باشید، ORDER BY باید ترتیبی یکتا و قطعی (deterministic) ایجاد کند. در غیر این صورت، ممکن است بهینه‌ساز کوئری در اجراهای مختلف، یک ردیف را در بیش از یک صفحه نمایش دهد.

۵.۲ حذف ردیف‌های تکراری با DISTINCT

معمولاً هر ردیف در یک جدول باید یکتا باشد. اما وقتی فقط زیرمجموعه‌ای از ستون‌ها را انتخاب می‌کنید، ممکن است چند ردیف از نظر مقادیر انتخاب‌شده با هم یکسان باشند، حتی اگر ردیف‌های اصلی جدول یکتا بودند. برای مثال، فرض کنید جدول تأمین‌کنندگان (Supplier) دارید که هر شهر/استان فقط یک تأمین‌کننده دارد. اما اگر فقط شهر و کشور را انتخاب کنید، ممکن است چند شهر تکراری در خروجی دیده شود:

```

1 SELECT City, CountryRegion
2 FROM Production.Supplier
3 ORDER BY CountryRegion, City;

```

به طور پیش فرض، عبارت SELECT یک کلمه ی کلیدی ضمنی به نام ALL دارد که همه ی ردیف ها را (حتی تکراری ها) برمی گرداند:

```

1 SELECT ALL City, CountryRegion
2 FROM Production.Supplier
3 ORDER BY CountryRegion, City;

```

برای حذف ردیف های تکراری، از کلمه ی کلیدی DISTINCT استفاده کنید:

```

1 SELECT DISTINCT City, CountryRegion
2 FROM Production.Supplier
3 ORDER BY CountryRegion, City;

```

با DISTINCT، فقط یک نمونه از هر ترکیب یکتای مقادیر موجود در لیست SELECT برگردانده می شود.

۶.۲ فیلتر کردن داده با WHERE

ساده ترین حالت SELECT (فقط با SELECT و FROM) همه ی ردیف های جدول را ارزیابی می کند. با اضافه کردن عبارت WHERE، شرط هایی تعریف می کنید که مشخص می کنند کدام ردیف ها باید در نتیجه باقی بمانند.

۱.۶.۲ ساختار WHERE

عبارت WHERE از یک یا چند شرط جستجو تشکیل شده که هر کدام برای هر ردیف باید TRUE، FALSE یا unknown (نامشخص) ارزیابی شود. فقط ردیف هایی که شرط برایشان TRUE باشد، در نتیجه باقی می ماند. به هر شرط، یک predicate گفته می شود. عملگرهای مقایسه ای پایه:

معنی	عملگر
برابر	=
نابرابر	<>
بزرگ تر از	>
بزرگ تر یا مساوی	>=
کوچک تر از	<
کوچک تر یا مساوی	<=

مثال:

```

1 SELECT ProductCategoryID AS Category, ProductName
2 FROM Production.Product
3 WHERE ProductCategoryID = 2;

```

```

1 SELECT ProductCategoryID AS Category, ProductName
2 FROM Production.Product
3 WHERE ListPrice < 10.00;

```

۲.۶.۲ NULL NOT IS / NULL IS

برای فیلتر بر اساس مقدار، NULL نمی توانید از = استفاده کنید (چون NULL با هیچ چیز، حتی خودش، برابر نیست). باید از IS NULL یا IS NOT NULL استفاده کنید:

```

1 SELECT ProductCategoryID AS Category, ProductName
2 FROM Production.Product
3 WHERE ProductName IS NOT NULL;

```

۳.۶.۲ ترکیب چند شرط با AND و OR

می‌توانید چند شرط را با AND و OR و در صورت نیاز پرانتز ترکیب کنید:

• با AND، همه‌ی شرط‌ها باید TRUE باشند.

• با OR، کافی‌ست یکی (یا هر دو) از شرط‌ها TRUE باشد.

```
1 SELECT ProductCategoryID AS Category, ProductName
2 FROM Production.Product
3 WHERE ProductCategoryID = 2
4     AND ListPrice < 10.00;
```

نکته‌ی مهم: AND قبل از OR پردازش می‌شود، مگر اینکه با پرانتز ترتیب اولویت را عوض کنید. بهترین روش این است که هرگاه بیش از دو شرط دارید، حتماً از پرانتز استفاده کنید تا هم منطق درست باشد و هم کد خواناتر شود:

```
1 SELECT ProductCategoryID AS Category, ProductName
2 FROM Production.Product
3 WHERE (ProductCategoryID = 2 OR ProductCategoryID = 3)
4     AND (ListPrice < 10.00);
```

۴.۶.۲ عملگرهای مقایسه‌ای مفید

IN — میان‌بری برای چند شرط تساوی روی یک ستون که با OR به هم وصل شده‌اند:

```
1 -- instead of this:
2 WHERE ProductCategoryID = 2 OR ProductCategoryID = 3 OR
3     ProductCategoryID = 4
4 -- you can write:
5 WHERE ProductCategoryID IN (2, 3, 4);
```

استفاده از IN خواناتر است و از نظر کارایی هم تفاوتی با نوشتن OR های متعدد ندارد.
BETWEEN — میان‌بری برای فیلتر بر اساس یک بازه‌ی بالا و پایین (شامل خود مرزها):

```
1 -- equivalent to:
2 WHERE ListPrice >= 1.00 AND ListPrice <= 10.00
3
4 -- can be written as:
5 WHERE ListPrice BETWEEN 1.00 AND 10.00;
```

BETWEEN برای فیلتر روی تاریخ هم بسیار کاربردی است:

```
1 SELECT ProductName, ModifiedDate
2 FROM Production.Product
3 WHERE ModifiedDate BETWEEN '2012-01-01' AND '2012-12-31';
```

نکته‌ی مهم درباره‌ی تاریخ و زمان: اگر ستون شما شامل بخش زمان (time) هم باشد، '2012-01-01' BETWEEN '2012-12-31' مقادیری را که در تاریخ ۳۱ دسامبر ولی با ساعت بعد از نیمه‌شب ثبت شده‌اند، شامل نمی‌شود؛ چون '2012-12-31' معادل '2012-12-31 00:00:00.000' در نظر گرفته می‌شود. برای پوشش کامل روز آخر، باید ساعت را هم صریح مشخص کنید:

```
1 WHERE ModifiedDate BETWEEN '2012-01-01 00:00:00.000' AND '2012-12-31
    23:59:59.999';
```

یا از عملگرهای مقایسه‌ای ساده استفاده کنید که این مشکل را ندارند:

```
1 WHERE ModifiedDate >= '2012-01-01' AND ModifiedDate < '2013-01-01';
```

LIKE — فقط برای داده‌ی متنی؛ به شما اجازه می‌دهد از wildcard (کاراکتر جایگزین) و الگوهای شبه-regex استفاده کنید:

- % هر رشته‌ای با طول ۰ یا بیشتر را نشان می‌دهد.

- _ دقیقاً یک کاراکتر را نشان می‌دهد.

- [0-9] یک بازه از کاراکترهای مجاز را نشان می‌دهد.

```
1 -- any product with "mountain" anywhere in its name
2 SELECT Name, ListPrice
3 FROM SalesLT.Product
4 WHERE Name LIKE '%mountain%';
5
6 -- exactly one character after "Mountain Bike Socks, "
7 SELECT ProductName, ListPrice
8 FROM SalesLT.Product
9 WHERE ProductName LIKE 'Mountain Bike Socks, _';
10
11 -- more complex pattern: Mountain- and three digits, space, any
12   string, comma, space, two digits
13 SELECT ProductName, ListPrice
14 FROM SalesLT.Product
15 WHERE ProductName LIKE 'Mountain-[0-9][0-9][0-9] %, [0-9][0-9]';
```

۷.۲ جمع‌بندی و نکات طلایی

- بدون ORDER BY، هیچ ترتیبی برای نتایج تضمین نمی‌شود.
- ORDER BY می‌تواند بر اساس نام ستون، alias، موقعیت عددی یا حتی ستون‌های خارج از SELECT (اگر DISTINCT نباشد) مرتب کند؛ جهت هرکدام هم می‌تواند مستقل باشد (DESC/ASC).
- TOP (N) همیشه به یک ORDER BY معنادار وابسته است؛ در غیر این صورت انتخاب ردیف‌ها تصادفی به نظر می‌رسد. برای شامل کردن مقادیر برابر با آخرین ردیف، از WITH TIES استفاده کنید.
- OFFSET ... FETCH NEXT ... ROWS ONLY بهترین ابزار برای پیاده‌سازی صفحه‌بندی است؛ فراموش نکنید ORDER BY باید ترتیب یکتا و قطعی ایجاد کند.
- SELECT DISTINCT ردیف‌های تکراری در ستون‌های انتخاب‌شده را حذف می‌کند (برخلاف حالت پیش‌فرض SE-LECT ALL).
- WHERE با predicate هایی که unknown/FALSE/TRUE برمی‌گردانند، ردیف‌ها را فیلتر می‌کند. برای فیلتر NULL از IS NOT NULL/IS NULL استفاده کنید، نه =.
- در ترکیب چند شرط، همیشه با پرانتز اولویت AND و OR را صریح مشخص کنید.
- IN جایگزین چند شرط تساوی با OR است؛ BETWEEN جایگزین بازه‌ی بین دو مقدار است؛ LIKE برای الگوهای متنی با wildcard به کار می‌رود.

این نوشته بر اساس مطالب مائول رسمی میکروسافت لرن تهیه شده است: [Learn Microsoft — T-SQL in results filter and Sort](#)

۳ JOIN ها در T-SQL

این فصل خلاصه‌ای ساده شده و به زبان فارسی از مبحث JOIN در T-SQL است که بر اساس مسیر یادگیری میکروسافت -Query and mod-Combine multiple tables with JOINS in T-SQL ify data with Transact-SQL و به طور مشخص مائول T-SQL تهیه شده و برای یادگیری شخصی بازنویسی شده است.

۱.۳ JOIN چیست و چرا به آن نیاز داریم؟

در یک دیتابیس رابطه‌ای، داده‌ها معمولاً در چند جدول جدا از هم ذخیره می‌شوند (مثلاً یک جدول برای مشتریان و یک جدول برای سفارش‌ها). وقتی بخواهیم اطلاعاتی را که در دو یا چند جدول پخش شده با هم ترکیب کنیم، از عملگر JOIN استفاده می‌کنیم. به بیان ساده، JOIN به ما اجازه می‌دهد ردیف‌های یک جدول را بر اساس یک شرط منطقی (معمولاً برابری یک ستون مشترک، مثل کلید خارجی) به ردیف‌های جدول دیگر وصل کنیم. می‌توان نتیجه‌ی یک JOIN را مثل یک نمودارون (Venn Diagram) تصور کرد: دو مجموعه (دو جدول) داریم و بسته به نوع JOIN، بخش‌های مشترک یا غیرمشترک آن‌ها را انتخاب می‌کنیم.

۲.۳ عبارت FROM و جدول مجازی

نکته‌ی مهمی که کمتر به آن توجه می‌شود این است: در پردازش منطقی یک SELECT عبارت FROM اولین بخشی است که اجرا می‌شود (قبل از SELECT، WHERE و غیره).

- FROM می‌تواند یک جدول یا چند جدول را به عنوان منبع داده معرفی کند.
 - خروجی FROM یک **جدول مجازی (Virtual Table)** است؛ یعنی یک جدول واقعی و فیزیکی در دیتابیس ساخته نمی‌شود، بلکه Server SQL به صورت منطقی این نتیجه را در حافظه نگه می‌دارد تا مراحل بعدی (مثل WHERE روی آن اعمال شوند).
 - هر عملگر JOIN که به FROM اضافه می‌کنید، یا ردیف اضافه می‌کند یا ردیف حذف می‌کند از این جدول مجازی.
- این دیدگاه کمک می‌کند بفهمیم چرا ترتیب نوشتن JOIN ها و شرط‌ها روی نتیجه‌ی نهایی تأثیر می‌گذارد.

۳.۳ دو نوع نحو نوشتن JOIN (تاریخچه)

۱.۳.۳ نحو قدیمی (ANSI-۸۹) SQL

در استاندارد قدیمی، جدول‌ها فقط با کاما در FROM پشت سر هم نوشته می‌شدند و شرط اتصال آن‌ها در WHERE قرار می‌گرفت:

```
1 SELECT c.CustomerID, o.OrderDate
2 FROM dbo.Customers AS c, dbo.Orders AS o
3 WHERE c.CustomerID = o.CustomerID;
```

این روش هنوز در Server SQL کار می‌کند، اما توصیه نمی‌شود، چون:

- در JOIN های پیچیده، خواندن و نگهداری کوئری سخت می‌شود.
- اگر به اشتباه WHERE را فراموش کنید، به جای یک JOIN معنادار، یک حاصل ضرب دکارتی (تعداد ردیف بسیار زیاد و اشتباه) دریافت می‌کنید.

۲.۳.۳ نحو جدید و استاندارد (ANSI-۹۲) SQL

از این استاندارد به بعد، کلمات کلیدی JOIN و ON معرفی شدند. شرط اتصال جدول‌ها به صورت واضح در ON نوشته می‌شود:

```
1 SELECT c.CustomerID, o.OrderDate
2 FROM dbo.Customers AS c
3 JOIN dbo.Orders AS o
4   ON c.CustomerID = o.CustomerID;
```

نکته: در این نحو، اگر کلمه JOIN را بنویسید ولی ON را فراموش کنید، SQL Server خطا می‌دهد (مگر اینکه واقعاً از CROSS JOIN استفاده کرده باشید). به همین دلیل احتمال ساختن ناخواسته‌ی حاصل ضرب دکارتی در این روش خیلی کمتر است. همیشه از این نحو استفاده کنید.

۴.۳ حاصل ضرب دکارتی (Cartesian Product)

در ریاضی، حاصل ضرب دکارتی یعنی ترکیب هر عضو یک مجموعه با هر عضو مجموعه‌ی دیگر. مثلاً اگر مجموعه‌ی اول ۲ عضو و مجموعه‌ی دوم ۳ عضو داشته باشد، حاصل ضرب دکارتی آن‌ها ۶ عضو (۳×۲) خواهد بود. در دیتابیس، اگر یک جدول ۱۰ ردیف و جدول دیگر ۱۰۰ ردیف داشته باشد، حاصل ضرب دکارتی آن‌ها ۱۰۰۰ ردیف است (هر ردیف جدول اول با هر ردیف جدول دوم ترکیب می‌شود). نکته‌ی جالب: از نظر منطقی، هر JOIN در پس‌زمینه از یک حاصل ضرب دکارتی شروع می‌شود و بعد با شرط ON فیلتر می‌شود (هرچند بهینه‌ساز SQL Server در عمل لزوماً این کار را به همین شکل فیزیکی انجام نمی‌دهد؛ این فقط یک مدل ذهنی مفید است). حاصل ضرب دکارتی معمولاً نتیجه‌ی مطلوب ما نیست، مگر در کاربردهای خاص مثل تولید داده‌ی تستی.

۵.۳ INNER JOIN

پراکندترین نوع JOIN، به خصوص در دیتابیس‌های نرمال شده، INNER JOIN فقط ردیف‌هایی را برمی‌گرداند که در هر دو جدول با شرط ON مطابقت داشته باشند.

۱.۵.۳ مراحل پردازش منطقی

```
1 SELECT emp.FirstName, ord.Amount
2 FROM HR.Employees AS emp
3 JOIN Sales.Orders AS ord
4     ON emp.EmployeeID = ord.EmployeeID;
```

پردازش این کوئری به این ترتیب انجام می‌شود:

۱. جدول HR.Employees با نام مستعار emp به عنوان یکی از منابع داده معرفی می‌شود.
۲. جدول Sales.Orders با نام مستعار ord به عنوان منبع دوم معرفی می‌شود؛ چون کلمه INNER نوشته نشده، پیش‌فرض همان INNER JOIN است.
۳. به صورت منطقی، یک حاصل ضرب دکارتی بین دو جدول ساخته می‌شود.
۴. با شرط ON emp.EmployeeID = ord.EmployeeID، فقط ردیف‌هایی نگه داشته می‌شوند که مقدار EmployeeID در هر دو جدول برابر باشد.
۵. نتیجه به مرحله‌ی بعدی (SELECT) تحویل داده می‌شود.

نتیجه‌ی نهایی: فقط کارمندانی که حداقل یک سفارش دارند، همراه با مبلغ سفارش‌شان نمایش داده می‌شوند. کارمندانی که سفارشی ثبت نکرده‌اند، از نتیجه حذف می‌شوند.

۲.۵.۳ نحو صریح (توصیه شده)

هرچند کلمه‌ی INNER اختیاری است، بهتر است همیشه صریح بنویسید تا خوانایی کد بالا برود:

```
1 SELECT emp.FirstName, ord.Amount
2 FROM HR.Employees AS emp
3 INNER JOIN Sales.Orders AS ord
4     ON emp.EmployeeID = ord.EmployeeID;
```

۳.۵.۳ نکات مهم INNER JOIN

- همیشه از **alias** (نام مستعار) برای جدول‌ها استفاده کنید؛ هم در SELECT و هم در ON.
- JOIN می‌تواند روی یک ستون یا چند ستون همزمان انجام شود. وقتی چند ستون با هم مقایسه می‌شوند، به آن **composite join** (جوین ترکیبی) می‌گویند.
- در INNER JOIN، ترتیب نوشتن جدول‌ها در FROM از نظر نتیجه‌ی نهایی و بهینه‌سازی داخلی Server SQL اهمیتی ندارد؛ اما از نظر مفهومی، JOIN‌ها از چپ به راست ارزیابی می‌شوند.
- برای هر جفت جدولی که می‌خواهید به هم وصل کنید، یک بار کلمه‌ی JOIN بنویسید. مثلاً برای وصل کردن ۳ جدول، به دو بار JOIN نیاز دارید.

۴.۵.۳ مثال: JOIN بیش از دو جدول

```
1 SELECT od.OrderID, m.ModelName, p.ProductName, od.Quantity
2 FROM Production.Products AS p
3 INNER JOIN Production.ProductModels AS m
4     ON p.ProductModelID = m.ProductModelID
5 INNER JOIN Sales.OrderDetails AS od
6     ON p.ProductID = od.ProductID
7 ORDER BY od.OrderID;
```

در اینجا ابتدا Products به ProductModels وصل می‌شود، سپس نتیجه‌ی آن به OrderDetails وصل می‌شود. SQL Server خودش ترتیب اجرای بهینه را انتخاب می‌کند.

۶.۳ OUTER JOIN (LEFT / RIGHT / FULL)

برخلاف INNER JOIN که فقط ردیف‌های مطابق را نگه می‌دارد، OUTER JOIN علاوه بر ردیف‌های مطابق، ردیف‌هایی را هم که در یکی از جدول‌ها مطابقت پیدا نکرده‌اند، نمایش می‌دهد (با مقدار NULL برای ستون‌های جدولی که مطابقتی نداشته).

۱.۶.۳ مقایسه با INNER JOIN

```
1 -- only employees who have orders
2 SELECT emp.FirstName, ord.Amount
3 FROM HR.Employees AS emp
4 INNER JOIN Sales.Orders AS ord
5     ON emp.EmployeeID = ord.EmployeeID;
```

```
1 -- all employees; those without orders are shown too (Amount = NULL)
2 SELECT emp.FirstName, ord.Amount
3 FROM HR.Employees AS emp
4 LEFT OUTER JOIN Sales.Orders AS ord
5     ON emp.EmployeeID = ord.EmployeeID;
```

در حالت LEFT OUTER JOIN، همه‌ی ردیف‌های جدول سمت چپ (HR.Employees) حفظ می‌شوند، چه با جدول سمت راست مطابقت داشته باشند چه نه.

۲.۶.۳ انواع JOIN OUTER

p۰.۲۸ p۰.۵۸
نوع چه ردیف‌هایی حفظ می‌شوند
LEFT OUTER JOIN همه‌ی ردیف‌های جدول چپ + ردیف‌های مطابق از جدول راست
RIGHT OUTER JOIN همه‌ی ردیف‌های جدول راست + ردیف‌های مطابق از جدول چپ
FULL OUTER JOIN همه‌ی ردیف‌های هر دو جدول (چه مطابق باشند چه نه)

نوشتن کلمه‌ی OUTER اختیاری است؛ یعنی LEFT JOIN همان LEFT OUTER JOIN است.

۳.۶.۳ نکات مهم OUTER JOIN

- مثل INNER JOIN، بهتر است همیشه alias بگذارید.
- OUTER JOIN هم می‌تواند روی یک یا چند ستون انجام شود.
- برخلاف INNER JOIN، در OUTER JOIN ترتیب جدول‌ها در FROM اهمیت دارد، چون مشخص می‌کند کدام جدول «چپ» و کدام «راست» است و همین تعیین می‌کند از LEFT استفاده کنید یا RIGHT.
- وقتی چند جدول را با OUTER JOIN به هم وصل می‌کنید، باید مراقب NULL ها باشید؛ اگر نتیجه‌ی یک OUTER JOIN (که شامل NULL است) دوباره با جدول سوم JOIN شود، ممکن است شرط JOIN دوم آن ردیف‌های NULL را حذف کند.
- برای نمایش فقط ردیف‌هایی که مطابقتی ندارند (یعنی فقط تفاوت دو مجموعه)، بعد از OUTER JOIN یک شرط WHERE ... IS NULL اضافه کنید:

```
1 -- only employees who have no orders at all
2 SELECT emp.FirstName
3 FROM HR.Employees AS emp
4 LEFT OUTER JOIN Sales.Orders AS ord
5     ON emp.EmployeeID = ord.EmployeeID
6 WHERE ord.OrderID IS NULL;
```

- FULL OUTER JOIN خیلی کم استفاده می‌شود.
- بدون ORDER BY، هیچ تضمینی درباره‌ی ترتیب برگشت ردیف‌ها وجود ندارد (نمی‌دانید ردیف‌های مطابق زودتر می‌آیند یا ردیف‌های بدون مطابقت).

۷.۳ CROSS JOIN

CROSS JOIN همان حاصل ضرب دکارتی است، اما این بار به صورت عمدی و صریح نوشته می‌شود. هیچ شرط تطبیقی بین دو جدول وجود ندارد، بنابراین همه‌ی ترکیب‌های ممکن بین ردیف‌های دو جدول تولید می‌شود.

```
1 SELECT emp.FirstName, prd.ProductName
2 FROM HR.Employees AS emp
3 CROSS JOIN Production.Products AS prd;
```

اگر جدول کارمندان ۵ ردیف و جدول محصولات ۲۰ ردیف داشته باشد، نتیجه ۱۰۰ ردیف خواهد بود (هر کارمند با هر محصول).

۱.۷.۳ نکات مهم CROSS JOIN

- چون هیچ تطبیقی انجام نمی‌شود، نباید و نمی‌توان ON نوشت؛ نوشتن ON همراه CROSS JOIN خطا می‌دهد.
- در نحو ANSI-۹۲، فقط کافی است دو جدول را با کلمه‌ی CROSS JOIN از هم جدا کنید.
- ۲.۷.۳ چه زمانی واقعاً به CROSS JOIN نیاز داریم؟
- ساختن یک جدول از اعداد یا تاریخ‌ها (مثلاً همه‌ی روزهای یک بازه‌ی زمانی).
- تولید حجم زیادی داده‌ی تستی؛ حتی یک جدول ۱۰۰ ردیفی وقتی با خودش CROSS JOIN شود، می‌تواند ۱۰,۰۰۰ ردیف تولید کند.

۸.۳ SELF JOIN

گاهی لازم است ردیف‌های یک جدول را با ردیف‌های دیگری از همان جدول مقایسه کنیم. نمونه‌ی کلاسیک: جدول کارمندان که هر کارمند یک ManagerID دارد و مدیر او هم خودش یک کارمند در همان جدول است.

ManagerID	FirstName	EmployeeID
NULL	دانا	1
1	آیدا	2
1	رزا	3
3	نیما	4

برای گرفتن نام هر کارمند در کنار نام مدیرش، جدول را دوبار در کوئری معرفی می‌کنیم و با alias های متفاوت از هم جدایشان می‌کنیم:

```

1 SELECT emp.FirstName AS Employee ,
2       mgr.FirstName AS Manager
3 FROM HR.Employees AS emp
4 LEFT OUTER JOIN HR.Employees AS mgr
5     ON emp.ManagerID = mgr.EmployeeID;
```

نتیجه:

Manager	Employee
NULL	دانا
دانا	آیدا
دانا	رزا
رزا	نیما

چرا از **LEFT OUTER JOIN** استفاده شد؟ چون مدیرعامل شرکت (دانا) خودش ManagerID ندارد. اگر از **INNER JOIN** استفاده می‌کردیم، ردیف دانا به‌طور کامل از نتیجه حذف می‌شد. با **LEFT OUTER JOIN**، دانا هم در نتیجه می‌ماند و ستون Manager برایش NULL می‌شود.

۱.۸.۳ نکات مهم SELF JOIN

- جدول را دو بار در **FROM** معرفی کنید و با **INNER** یا **OUTER JOIN** (بسته به نیاز) به هم وصل کنید.
- حتماً برای دو نمونه از جدول، **alias** های متفاوت بگذارید (وگرنه **SQL Server** نمی‌تواند تشخیص دهد منظورشان کدام نمونه است).
- شرط **ON** را طوری بنویسید که ستون‌های یک نمونه را با ستون‌های نمونه‌ی دیگر مقایسه کند.
- کاربرد دیگر **SELF JOIN**: مقایسه‌ی ردیف‌هایی که در دو زمان مختلف ثبت شده‌اند (مثلاً یک ردیف که زمان شروع را ذخیره کرده و ردیف دیگری در همان جدول که زمان پایان را دارد).

۹.۳ جمع‌بندی و نکات طلایی

- همیشه از نحو استاندارد **JOIN ... ON** استفاده کنید، نه روش قدیمی **FROM**.
- **INNER JOIN**: فقط ردیف‌های مطابق در هر دو جدول.
- **LEFT/RIGHT/FULL OUTER JOIN**: همه‌ی ردیف‌های یک طرف (یا هر دو طرف در **FULL**) حتی بدون مطابقت؛ ستون‌های بدون مطابقت مقدار **NULL** می‌گیرند.
- **CROSS JOIN**: بدون هیچ شرطی، همه‌ی ترکیب‌های ممکن (حاصل ضرب دکارتی).
- **SELF JOIN**: وصل کردن یک جدول به خودش با دو **alias** متفاوت.

- همیشه برای جدول‌ها alias بگذارید و در ON از آن‌ها استفاده کنید.
- در OUTER JOIN، ترتیب جدول‌ها مهم است؛ در INNER JOIN مهم نیست.
- بدون ORDER BY، ترتیب ردیف‌های خروجی تضمین شده نیست.

منبع

این نوشته بر اساس مطالب مائول رسمی مایکروسافت لن تهیه شده است: [Microsoft — T-SQL in JOINs with tables multiple Combine](#)
[Learn](#)

۴ نوشتن Subquery ها در T-SQL

این فصل خلاصه‌ای ساده شده و به زبان فارسی از مائول چهارم مسیر یادگیری مایکروسافت Query and modify data with Transact-SQL است؛ مائول Write Subqueries in T-SQL که بعد از مبحث JOIN می‌آید و یاد می‌دهد چطور یک کوئری SELECT را داخل کوئری دیگری بنویسیم.

۱.۴ Subquery چیست؟

Subquery (زیرکوئری یا کوئری تودرتو) یعنی یک دستور SELECT که داخل یک کوئری دیگر نوشته شده باشد. با تودرتو کردن کوئری‌ها می‌توانید سؤالات پیچیده‌تری از دیتابیس پرسید که نوشتن آن‌ها فقط با یک SELECT ساده ممکن نیست. در اصطلاح:

- کوئری‌ای که داخل کوئری دیگر نوشته شده را **کوئری داخلی (Inner Query)** یا همان Subquery می‌نامند.
 - کوئری‌ای که Subquery داخل آن قرار گرفته را **کوئری بیرونی (Outer Query)** می‌نامند.
- هدف یک Subquery این است که یک نتیجه به کوئری بیرونی تحویل بدهد. بسته به شکل این نتیجه، Subquery به دو دسته تقسیم می‌شود:

- **Scalar Subquery (اسکالر):** فقط یک مقدار تکی برمی‌گرداند.
- **Multi-Valued Subquery (چندمقداری):** چیزی شبیه یک جدول تک‌ستونی برمی‌گرداند؛ یعنی می‌تواند چند ردیف داشته باشد، اما همیشه فقط یک ستون.

از زاویه‌ی دیگری هم می‌توان Subquery ها را دسته‌بندی کرد:

- **Self-Contained (مستقل):** هیچ وابستگی‌ای به کوئری بیرونی ندارد و می‌تواند به تنهایی هم اجرا شود.
- **Correlated (وابسته):** به یک یا چند ستون از کوئری بیرونی ارجاع می‌دهد و بدون کوئری بیرونی قابل اجرا نیست.

۲.۴ Subquery اسکالر (Scalar Subquery)

یک Subquery اسکالر، یک SELECT داخلی است که فقط یک مقدار تکی برمی‌گرداند. از این نوع Subquery می‌توان در هر جایی از کوئری بیرونی استفاده کرد که یک مقدار تکی مورد انتظار است؛ مثلاً در لیست SELECT، در WHERE، در HAVING یا حتی در FROM. همچنین می‌توان از آن در دستورات تغییر داده مثل UPDATE یا DELETE هم استفاده کرد.

۱.۲.۴ مثال: پیدا کردن جزئیات آخرین سفارش

فرض کنید می‌خواهیم جزئیات آخرین سفارشی که ثبت شده را پیدا کنیم؛ با این فرض که آخرین سفارش، همان سفارشی است که بیشترین SalesOrderID را دارد.

اول باید بیشترین مقدار SalesOrderID را پیدا کنیم:

```
1 SELECT MAX(SalesOrderID)
2 FROM Sales.SalesOrderHeader;
```

حالا برای گرفتن جزئیات همان سفارش، این کوئری را به عنوان Subquery داخل شرط WHERE کوئری دیگری قرار می‌دهیم:

```
1 SELECT SalesOrderID, ProductID, OrderQty
2 FROM Sales.SalesOrderDetail
3 WHERE SalesOrderID =
4     (SELECT MAX(SalesOrderID)
5      FROM Sales.SalesOrderHeader);
```

۲.۲.۴ نکات مهم نوشتن Subquery اسکالر

- Subquery همیشه باید داخل پرانتز نوشته شود.
- T-SQL از چند سطح تودرتویی پشتیبانی می‌کند (حتی تا ۳۲ سطح!)، اما در این مازول فقط روی حالت دوسطحی (یک Subquery داخل یک کوئری بیرونی) تمرکز می‌کنیم.
- اگر Subquery هیچ ردیفی برگرداند (یعنی نتیجه‌اش خالی باشد)، نتیجه‌ی آن NULL در نظر گرفته می‌شود. اگر احتمال می‌دهید در سناریوی شما ممکن است هیچ ردیفی برگردد، باید مطمئن شوید کوئری بیرونی‌تان بتواند این حالت NULL را به درستی مدیریت کند.
- کوئری داخلی معمولاً باید فقط یک ستون برگرداند؛ برگرداندن چند ستون در Subquery تقریباً همیشه خطاست، مگر اینکه Subquery با کلمه‌ی EXISTS معرفی شده باشد (که در ادامه توضیح می‌دهیم).

۳.۲.۴ استفاده در لیست SELECT

می‌توان از Subquery اسکالر مستقیماً در لیست SELECT هم استفاده کرد؛ برای مثال، مقایسه‌ی تعداد اقلام سفارش داده شده در آخرین سفارش با میانگین کل سفارش‌ها:

```
1 SELECT SalesOrderID, ProductID, OrderQty,
2     (SELECT AVG(OrderQty)
3      FROM SalesLT.SalesOrderDetail) AS AvgQty
4 FROM SalesLT.SalesOrderDetail
5 WHERE SalesOrderID =
6     (SELECT MAX(SalesOrderID)
7      FROM SalesLT.SalesOrderHeader);
```

۳.۴ Subquery چندمقداری (Multi-Valued Subquery)

یک Subquery چندمقداری، برخلاف حالت اسکالر، می‌تواند چند ردیف برگرداند (اما همچنان فقط یک ستون). این نوع Subquery معمولاً همراه عملگر IN استفاده می‌شود.

۱.۳.۴ مثال

پیدا کردن CustomerID و SalesOrderID همه‌ی سفارش‌هایی که توسط مشتریان کانادایی ثبت شده‌اند:

```
1 SELECT CustomerID, SalesOrderID
2 FROM Sales.SalesOrderHeader
3 WHERE CustomerID IN (
4     SELECT CustomerID
5     FROM Sales.Customer
6     WHERE CountryRegion = 'Canada');
```

اگر فقط کوئری داخلی را جدا اجرا کنید، یک ستون از مقادیر CustomerID برمی‌گردد — یک ردیف برای هر مشتری کانادایی.

۴.۴ در مقابل Subquery کدام JOIN را انتخاب کنیم؟

بسیاری از Subquery های چندمقداری را می‌توان با JOIN هم نوشت. برای مثال، همان کوئری بالا را با JOIN می‌توان این‌طور نوشت:

```
1 SELECT c.CustomerID, o.SalesOrderID
2 FROM Sales.Customer AS c
3 JOIN Sales.SalesOrderHeader AS o
4   ON c.CustomerID = o.CustomerID
5 WHERE c.CountryRegion = 'Canada';
```

پس چطور تصمیم بگیریم کدام روش را استفاده کنیم؟ چند نکته برای این تصمیم‌گیری:

- گاهی صرفاً به سلیقه و راحتی شما در نوشتن کوئری برمی‌گردد. بسیاری از Subquery هایی که به راحتی قابل تبدیل به JOIN هستند، در پس‌زمینه توسط SQL Server به همان شکل JOIN بهینه‌سازی می‌شوند؛ یعنی از نظر کارایی معمولاً تفاوتی وجود ندارد.
- محدودیت مهم:** وقتی از یک Subquery (کوئری تودرتو) استفاده می‌کنید، نتایجی که به کلاینت برمی‌گردد فقط می‌تواند شامل ستون‌های کوئری بیرونی باشد. پس اگر لازم است ستون‌هایی از هر دو جدول را برگردانید، باید حتماً از JOIN استفاده کنید.
- در مواردی که کوئری داخلی باید پردازش‌های پیچیده‌تری نسبت به یک واکنشی ساده انجام دهد، بازنویسی آن با JOIN می‌تواند سخت باشد. خیلی از توسعه‌دهندگان SQL ترجیح می‌دهند برای پردازش‌های پیچیده از Subquery استفاده کنند، چون امکان می‌دهد مسئله را به مراحل کوچک‌تر بشکنید.
- در برخی موارد خاص که باید وجود داشتن (existence) یک رابطه را بررسی کنید، استفاده از JOIN می‌تواند از نظر کارایی بهتر عمل کند.

۵.۴ Subquery مستقل Self-Contained در مقابل وابسته Correlated

تا اینجا مثال‌هایی از Subquery مستقل Self-Contained دیدیم؛ یعنی کوئری داخلی کاملاً مستقل از کوئری بیرونی است، فقط یک بار اجرا می‌شود و نتیجه‌اش را به کوئری بیرونی تحویل می‌دهد. T-SQL نوع دیگری هم دارد به نام **Correlated Subquery (وابسته)**: کوئری داخلی به یک یا چند ستون از کوئری بیرونی ارجاع می‌دهد و از نظر مفهومی، به ازای هر ردیف کوئری بیرونی، یک بار اجرا می‌شود.

۱.۵.۴ مثال: پیدا کردن آخرین سفارش هر مشتری

```
1 SELECT SalesOrderID, CustomerID, OrderDate
2 FROM SalesLT.SalesOrderHeader AS o1
3 WHERE SalesOrderID =
4   (SELECT MAX(SalesOrderID)
5    FROM SalesLT.SalesOrderHeader AS o2
6    WHERE o2.CustomerID = o1.CustomerID)
7 ORDER BY CustomerID, OrderDate;
```

در این مثال، کوئری داخلی به ستون CustomerID از کوئری بیرونی (که با alias نمونه‌ی o1 مشخص شده) ارجاع می‌دهد. برای هر ردیف در کوئری بیرونی، کوئری داخلی بیشترین SalesOrderID مربوط به همان مشتری را پیدا می‌کند، و کوئری بیرونی بررسی می‌کند آیا ردیف فعلی همان سفارش با بیشترین ID است یا نه.

۲.۵.۴ تفاوت‌های مهم با Subquery مستقل

- Subquery وابسته را نمی‌توان جدا از کوئری بیرونی اجرا کرد؛ همین موضوع تست و دیباگ کردن آن را سخت‌تر می‌کند.
- برخلاف Subquery مستقل که فقط یک بار اجرا می‌شود، Subquery وابسته به صورت منطقی چندبار اجرا می‌شود — یک بار برای هر ردیف کوئری بیرونی.

۳.۵.۴ نکات نوشتن Subquery وابسته

- کوئری بیرونی را طوری بنویسید که بتواند نتیجه‌ی درست را از کوئری داخلی بپذیرد: اگر کوئری داخلی اسکالر است، از عملگرهای تساوی و مقایسه (> < <= >=) استفاده کنید؛ اگر ممکن است چند مقدار برگرداند، از IN استفاده کنید. همیشه فکر کنید که اگر نتیجه NULL شد چطور باید مدیریت شود.
 - ستونی از کوئری بیرونی که قرار است در Subquery به آن ارجاع دهید را مشخص کنید، و برای جدول منبع آن در کوئری بیرونی یک alias بگذارید.
 - ستونی از جدول داخلی که باید با ستون کوئری بیرونی مقایسه شود را هم مشخص کنید و برای جدول منبع آن هم یک alias بگذارید.
 - کوئری داخلی را طوری بنویسید که مقدار ورودی از کوئری بیرونی را در شرط WHERE خودش استفاده کند.
- اسم «Correlated» (وابسته) دقیقاً از همین ارتباط بین مقدار بیرونی و ارجاع آن در کوئری داخلی گرفته شده است.

۶.۴ کار با EXISTS

علاوه بر گرفتن مقدار از یک T-SQL Subquery، روشی هم برای بررسی وجود داشتن نتیجه از یک کوئری دارد: عبارت EXISTS. EXISTS بررسی می‌کند آیا اصلاً ردیفی با شرایط مشخص وجود دارد یا نه — اما به جای برگرداندن خود ردیف‌ها، فقط TRUE یا FALSE برمی‌گرداند. این روش برای بررسی وجود داده، بدون هزینه‌ی اضافی واکشی و پردازش کامل نتایج، بسیار مفید است. از نظر مفهومی، EXISTS معادل این است که نتیجه را بگیرید، تعداد ردیف‌ها را بشمارید و آن را با صفر مقایسه کنید. این دو کوئری زیر همین کار را انجام می‌دهند — پیدا کردن مشتریانی که حداقل یک سفارش ثبت کرده‌اند:

روش اول، با: COUNT

```
1 SELECT CustomerID, CompanyName, EmailAddress
2 FROM SalesLT.Customer AS c
3 WHERE
4 (SELECT COUNT(*)
5  FROM SalesLT.SalesOrderHeader AS o
6  WHERE o.CustomerID = c.CustomerID) > 0;
```

روش دوم، با: EXISTS

```
1 SELECT CustomerID, CompanyName, EmailAddress
2 FROM SalesLT.Customer AS c
3 WHERE EXISTS
4 (SELECT *
5  FROM SalesLT.SalesOrderHeader AS o
6  WHERE o.CustomerID = c.CustomerID);
```

در روش اول، کوئری داخلی مجبور است همه‌ی سفارش‌های هر مشتری را بشمارد، فقط برای اینکه ثابت کند حداقل یکی وجود دارد. در روش دوم، به محض پیدا شدن اولین سفارش مرتبط، EXISTS مقدار TRUE برمی‌گرداند و دیگر لازم نیست بقیه‌ی ردیف‌ها هم شمارش شوند. نکته‌ی جالب دیگر: در فرم EXISTS، کوئری داخلی محدود به برگرداندن یک ستون نیست؛ به همین دلیل معمولاً از SELECT * استفاده می‌شود، چون مقدار واقعی ستون‌ها اصلاً اهمیتی ندارد — فقط وجود یا عدم وجود ردیف مهم است. از نظر پردازش منطقی، این دو فرم کوئری معادل هم هستند؛ اما از نظر کارایی، ممکن است بهینه‌ساز Server SQL رفتار متفاوتی با آن‌ها داشته باشد. بهتر است هر دو حالت را برای سناریوی خودتان تست کنید.

نکته‌ی مهم: اگر می‌خواهید یک Subquery مبتنی بر COUNT(*) را به فرم EXISTS تبدیل کنید، حتماً از SELECT * داخل آن استفاده کنید، نه SELECT COUNT(*). چون SELECT COUNT(*) همیشه (حتی وقتی نتیجه صفر است) یک ردیف برمی‌گرداند، و در نتیجه EXISTS همیشه TRUE خواهد شد!

۱.۶.۴ استفاده از NOT EXISTS

یکی دیگر از کاربردهای مفید EXISTS، منفی کردن آن با NOT است؛ مثلاً برای پیدا کردن مشتریانی که هیچ وقت سفارشی ثبت نکرده‌اند:

```
1 SELECT CustomerID, CompanyName, EmailAddress
2 FROM SalesLT.Customer AS c
3 WHERE NOT EXISTS
4     (SELECT *
5      FROM SalesLT.SalesOrderHeader AS o
6      WHERE o.CustomerID = c.CustomerID);
```

با این روش، Server SQL دیگر لازم نیست داده‌ی سفارش‌های مربوط به مشتریانی که واقعاً سفارش دارند را واکنشی کند. اگر یک CustomerID در جدول سفارش‌ها پیدا شود، NOT EXISTS بلافاصله FALSE می‌شود و ارزیابی سریع تمام می‌شود.

۲.۶.۴ نکات نوشتن EXISTS

- کلمه‌ی EXISTS باید بلافاصله بعد از WHERE بیاید؛ هیچ نام ستون یا عبارتی قبل از آن نمی‌آید، مگر اینکه NOT هم استفاده شده باشد.
- داخل Subquery، از SELECT * استفاده کنید؛ چون در نهایت هیچ ردیفی از Subquery واقعاً برگردانده نمی‌شود، پس نیازی نیست ستون خاصی مشخص کنید.

۷.۴ جمع‌بندی و نکات طلایی

- Subquery یعنی یک SELECT داخل SELECT دیگر؛ به SELECT داخلی «Inner Query» و به بیرونی «Outer Query» می‌گویند.
- Subquery Scalar یک مقدار تکی برمی‌گرداند و تقریباً هر جایی که یک مقدار انتظار می‌رود قابل استفاده است.
- Multi-Valued Subquery می‌تواند چند ردیف (ولی فقط یک ستون) برگرداند؛ معمولاً همراه IN استفاده می‌شود.
- خیلی از Subquery‌های چندمقداری را می‌توان معادل با JOIN نوشت؛ تفاوت اصلی این است که Subquery فقط اجازه می‌دهد ستون‌های کوئری بیرونی را برگردانید، در حالی که JOIN اجازه می‌دهد ستون از هر دو جدول برگردد.
- Self-Contained Subquery مستقل است و یک بار اجرا می‌شود؛ Correlated Subquery به کوئری بیرونی وابسته است و به ازای هر ردیف کوئری بیرونی، یک بار (منطقاً) اجرا می‌شود.
- برای بررسی صرف «وجود دارد یا نه» (بدون نیاز به خود مقادیر)، از EXISTS استفاده کنید — از $COUNT(*) > 0$ کارا تر است. برای پیدا کردن ردیف‌هایی که هیچ رکورد مرتبطی ندارند، از NOT EXISTS استفاده کنید.
- همیشه Subquery را داخل پرانتز بنویسید و به احتمال برگشتن NULL (وقتی هیچ ردیفی مطابقت ندارد) توجه کنید.

منبع

این نوشته بر اساس مطالب ماژول رسمی میکروسافت لرن تهیه شده است: [Learn Microsoft — T-SQL in Subqueries Write](#)

۵ توابع درون ساخت و GROUP BY

این فصل خلاصه‌ای ساده شده و به زبان فارسی از ماژول پنجم مسیر یادگیری میکروسافت Query and modify data with Transact-SQL است؛ ماژول Use built-in functions and GROUP BY in Transact-SQL که بعد از مبحث Subquery می‌آید.

۱.۵ دسته‌بندی توابع درون ساخت T-SQL

T-SQL تعداد زیادی تابع آماده (Built-in Function) دارد؛ از توابعی که نوع داده را تبدیل می‌کنند، تا توابعی که روی گروه‌هایی از ردیف‌ها تحلیل و تجمیع انجام می‌دهند. توابع T-SQL را می‌توان به این دسته‌ها تقسیم کرد:

p۰.۶۲ p۰.۲۴
دسته توضیح
Scalar (اسکالر) روی یک ردیف عمل می‌کند و یک مقدار تکی برمی‌گرداند.
Logical (منطقی) چند مقدار را با هم مقایسه می‌کند تا یک خروجی واحد تعیین کند.
Ranking (رتبه‌بندی) روی یک مجموعه (partition) از ردیف‌ها عمل می‌کند.
Rowset یک جدول مجازی برمی‌گرداند که می‌توان در عبارت FROM از آن استفاده کرد.
Aggregate (تجمیعی) یک یا چند مقدار ورودی می‌گیرد و یک مقدار خلاصه‌شده برمی‌گرداند.

۲.۵ توابع اسکالر (Scalar Functions)

توابع اسکالر یک مقدار تکی برمی‌گردانند و معمولاً روی یک ردیف از داده کار می‌کنند. تعداد ورودی‌هایی که می‌گیرند می‌تواند صفر باشد (مثل GETDATE)، یک (مثل UPPER) یا چند تا (مثل ROUND). چون همیشه یک مقدار تکی برمی‌گردانند، هرجایی که یک مقدار تکی مورد نیاز باشد قابل استفاده‌اند — بیشتر در عبارت‌های SELECT و شرط‌های WHERE، و همچنین در بخش SET دستور UPDATE.

۱.۲.۵ دو نکته‌ی مهم درباره‌ی توابع اسکالر

- قطعیت (Determinism):** اگر یک تابع برای ورودی و وضعیت یکسان دیتابیس، همیشه همان مقدار را برگرداند، به آن قطعی (deterministic) می‌گویند. مثلاً ROUND(1.1, 0) همیشه 1.0 برمی‌گرداند. اما بسیاری از توابع غیرقطعی (nondeterministic) هستند؛ مثلاً GETDATE() که تاریخ و ساعت فعلی را برمی‌گرداند. نتایج توابع غیرقطعی قابل ایندکس‌گذاری نیستند و همین موضوع روی توانایی بهینه‌ساز کوئری برای انتخاب یک نقشه‌ی اجرای خوب تأثیر می‌گذارد.
- Collation (ترتیب مرتب‌سازی/مقایسه‌ی متن):** وقتی از توابعی استفاده می‌کنید که روی داده‌ی متنی کار می‌کنند، باید بدانید کدام collation استفاده می‌شود — بعضی توابع از collation مقدار ورودی استفاده می‌کنند و بعضی دیگر، اگر ورودی مشخصی نداشته باشند، از collation خود دیتابیس استفاده می‌کنند.

۲.۲.۵ نمونه‌هایی از توابع اسکالر پرکاربرد

مستندات رسمی Server SQL بیش از ۲۰۰ تابع اسکالر را در دسته‌های مختلف فهرست می‌کند: پیکربندی، تبدیل نوع داده، CURSOR، تاریخ و زمان، ریاضی، متادیتا، امنیت، رشته و... در ادامه چند نمونه‌ی رایج را می‌بینیم.

توابع تاریخ و زمان:

```
1 SELECT SalesOrderID,
2    OrderDate,
3    YEAR(OrderDate) AS OrderYear,
4    DATENAME(mm, OrderDate) AS OrderMonth,
5    DAY(OrderDate) AS OrderDay,
6    DATENAME(dw, OrderDate) AS OrderWeekDay,
7    DATEDIFF(yy, OrderDate, GETDATE()) AS YearsSinceOrder
8 FROM Sales.SalesOrderHeader;
```

توابع ریاضی:

```
1 SELECT TaxAmt,
2    ROUND(TaxAmt, 0) AS Rounded,
3    FLOOR(TaxAmt) AS Floor,
4    CEILING(TaxAmt) AS Ceiling,
5    SQUARE(TaxAmt) AS Squared,
6    SQRT(TaxAmt) AS Root,
7    LOG(TaxAmt) AS Log,
8    TaxAmt * RAND() AS Randomized
9 FROM Sales.SalesOrderHeader;
```

توابع رشته‌ای:

```

1 SELECT CompanyName ,
2         UPPER(CompanyName) AS UpperCase ,
3         LOWER(CompanyName) AS LowerCase ,
4         LEN(CompanyName) AS Length ,
5         REVERSE(CompanyName) AS Reversed ,
6         CHARINDEX(' ', CompanyName) AS FirstSpace ,
7         LEFT(CompanyName, CHARINDEX(' ', CompanyName)) AS FirstWord ,
8         SUBSTRING(CompanyName, CHARINDEX(' ', CompanyName) + 1, LEN(
          CompanyName)) AS RestOfName
9 FROM Sales.Customer;

```

۳.۲.۵ توابع منطقی

توابع منطقی به شما کمک می‌کنند تصمیم بگیرید کدام یک از چند مقدار باید برگردانده شود؛ یک عبارت ورودی را ارزیابی می‌کنند و بر اساس نتیجه، مقدار مناسب را برمی‌گردانند.

IIF — یک عبارت بولی (True/False) را ارزیابی می‌کند و بسته به نتیجه، یکی از دو مقدار مشخص شده را برمی‌گرداند:

```

1 SELECT AddressType ,
2         IIF(AddressType = 'Main Office', 'Billing', 'Mailing') AS
          UseAddressFor
3 FROM Sales.CustomerAddress;

```

CHOOSE — یک عبارت عددی را ارزیابی می‌کند و مقدار متناظر با آن موقعیت (شماره‌گذاری از ۱) را از یک لیست برمی‌گرداند:

```

1 SELECT SalesOrderID, Status,
2        CHOOSE(Status, 'Ordered', 'Shipped', 'Delivered') AS OrderStatus
3 FROM Sales.SalesOrderHeader;

```

۳.۵ توابع رتبه‌بندی و Rowset

توابع رتبه‌بندی و Rowset برخلاف توابع اسکالر، یک مقدار تکی بر نمی‌گردانند؛ بلکه یک مجموعه از ردیف‌ها را به عنوان ورودی می‌گیرند و یک مجموعه از ردیف‌ها را به عنوان خروجی برمی‌گردانند.

۱.۳.۵ توابع رتبه‌بندی (Ranking)

این توابع محاسباتی را روی یک مجموعه‌ی تعریف شده توسط کاربر از ردیف‌ها انجام می‌دهند و شامل توابع رتبه‌بندی، افسست، تجمیعی و توزیعی هستند.

مثال: استفاده از RANK برای رتبه‌بندی محصولات بر اساس قیمت (گران‌ترین، رتبه‌ی ۱):

```

1 SELECT TOP 100 ProductID, Name, ListPrice,
2        RANK() OVER(ORDER BY ListPrice DESC) AS RankByPrice
3 FROM Production.Product AS p
4 ORDER BY RankByPrice;

```

۲.۳.۵ عبارت OVER و BY PARTITION

با عبارت OVER می‌توانید داده را به قسمت‌های مجزا (Partition) تقسیم کنید. برای مثال، محاسبه‌ی رتبه‌بندی قیمت محصولات در داخل هر دسته‌بندی (نه در کل جدول):

```

1 SELECT c.Name AS Category, p.Name AS Product, ListPrice,
2        RANK() OVER(PARTITION BY c.Name ORDER BY ListPrice DESC) AS
          RankByPrice
3 FROM Production.Product AS p
4 JOIN Production.ProductCategory AS c

```

```

5 ON p.ProductCategoryID = c.ProductcategoryID
6 ORDER BY Category, RankByPrice;

```

نکته: ممکن است چند ردیف رتبه‌ی یکسانی بگیرند و بعضی شماره‌ها در توالی رتبه‌ها رد شوند؛ این رفتار طبیعی تابع RANK است. اگر نمی‌خواهید رتبه‌های برابر (tie) داشته باشید، می‌توانید از توابع مشابه ROW_NUMBER یا NTILE، DENSE_RANK استفاده کنید که رفتار متفاوتی دارند.

۳.۳.۵ توابع Rowset

توابع Rowset یک جدول مجازی برمی‌گردانند که می‌توان از آن به عنوان منبع داده در عبارت FROM استفاده کرد. مهم‌ترین این توابع عبارت‌اند از OPENJSON، OPENXML، OPENROWSET، OPENQUERY، OPENDATASOURCE.

• OPENROWSET و OPENQUERY، OPENDATASOURCE به شما اجازه می‌دهند یک کوئری را به یک سرور دیتابیس دیگر (remote) بفرستید و نتیجه را بگیرید:

```

1 SELECT a.*
2 FROM OPENROWSET('SQLNCLI', 'Server=SalesDB;Trusted_Connection=yes;',
3     'SELECT Name, ListPrice
4     FROM AdventureWorks.Production.Product') AS a;

```

برای استفاده از سرورهای دیگر، باید تنظیمات پیشرفته‌ای در نمونه‌ی Server SQL خودتان فعال کنید.

• OPENJSON و OPENXML به شما اجازه می‌دهند داده‌ی ساختاریافته در قالب XML یا JSON را کوئری بگیرید و مقادیر آن را به صورت یک جدولی استخراج کنید.

بررسی کامل توابع Rowset خارج از حوصله‌ی این مآژول است.

۴.۵ توابع پنجره‌ای پیشرفته (Advanced Window Functions)

آنچه در بخش قبل با RANK دیدیم، فقط گوشه‌ای از توانایی‌های واقعی عبارت OVER بود. T-SQL خانواده‌ی کاملی از توابع پنجره‌ای دارد که به شما اجازه می‌دهند بدون نیاز به GROUP BY و بدون از دست دادن ردیف‌های اصلی، محاسباتی روی «پنجره‌ای» از ردیف‌های مرتبط با هر ردیف انجام دهید — چیزی که با یک SELECT ساده یا حتی یک Subquery معمولی به این سادگی و کارایی ممکن نیست.

۱.۴.۵ چرا اصلاً به توابع پنجره‌ای نیاز داریم؟

فرض کنید می‌خواهید برای هر سفارش، مبلغ آن را در کنار مبلغ سفارش قبلی همان مشتری نمایش دهید. اگر بخواهید این کار را با GROUP BY انجام دهید، مجبورید ردیف‌های اصلی را از دست بدهید (چون GROUP BY همیشه ردیف‌ها را در یک گروه خلاصه می‌کند). توابع پنجره‌ای این مشکل را حل می‌کنند: محاسبه را انجام می‌دهند، اما هیچ ردیفی را خلاصه یا حذف نمی‌کنند.

تفاوت کلیدی با GROUP BY: GROUP BY تعداد ردیف‌های خروجی را کم می‌کند (هر گروه یک ردیف). توابع پنجره‌ای (Window Functions) تعداد ردیف‌های خروجی را دست‌نخورده نگه می‌دارند و فقط یک ستون محاسبه‌شده‌ی اضافه به هر ردیف می‌افزایند.

۲.۴.۵ مروری بر ساختار عبارت OVER

ساختار کلی هر تابع پنجره‌ای به این شکل است:

```

1 <function_name> ( <arguments> ) OVER (
2     [ PARTITION BY <column_list> ]
3     [ ORDER BY <column_list> ]
4     [ <window frame clause> ]
5 )

```

• PARTITION BY (اختیاری): داده را به گروه‌های مجزا تقسیم می‌کند — شبیه GROUP BY، اما بدون خلاصه کردن ردیف‌ها. اگر ننویسید، کل نتیجه یک پارتیشن واحد در نظر گرفته می‌شود.

• **ORDER BY** (برای بعضی توابع الزامی): ترتیب ردیف‌ها را درون هر پارتیشن مشخص می‌کند. برای توابعی مثل ROW_NUMBER, RANK, LAG و LEAD الزامی است.

• **Window Frame** (اختیاری): چارچوب دقیق ردیف‌هایی که باید در محاسبه لحاظ شوند را مشخص می‌کند (در ادامه توضیح داده می‌شود).

۳.۴.۵ توابع رتبه‌بندی: تفاوت ROW_NUMBER, RANK, DENSE_RANK و NTILE

این چهار تابع همگی به هر ردیف یک عدد رتبه می‌دهند، اما رفتارشان با مقادیر برابر (tie) کاملاً متفاوت است. برای درک بهتر، این جدول محصولات را با قیمت‌های تکراری در نظر بگیرید:

```
1 SELECT
2     Name ,
3     ListPrice ,
4     ROW_NUMBER() OVER (ORDER BY ListPrice DESC) AS RowNum ,
5     RANK() OVER (ORDER BY ListPrice DESC) AS Rank ,
6     DENSE_RANK() OVER (ORDER BY ListPrice DESC) AS DenseRank ,
7     NTILE(4) OVER (ORDER BY ListPrice DESC) AS Quartile
8 FROM Production.Product ;
```

|p۰.۶۵|p۰.۲۰|

تابع رفتار

ROW_NUMBER() به هر ردیف یک شماره‌ی یکتا و بدون تکرار می‌دهد (۱، ۲، ۳، ...۴)، حتی اگر مقدار چند ردیف کاملاً برابر باشد.

کدام ردیف اول شماره بگیرد در صورت تساوی مقدار، تصادفی/ غیرقطعی است مگر یک ORDER BY کاملاً یکتا بدهید.

RANK() ردیف‌های با مقدار برابر، رتبه‌ی یکسان می‌گیرند؛ اما بعد از آن‌ها، رتبه‌ی بعدی جهش می‌کند (مثلاً اگر دو ردیف رتبه‌ی ۲ بگیرند،

ردیف بعدی رتبه‌ی ۴ می‌گیرد، نه ۳).

DENSE_RANK() دقیقاً مثل RANK، با این تفاوت که رتبه‌ی بعدی جهش نمی‌کند (بعد از دو ردیف با رتبه‌ی ۲، ردیف بعدی

رتبه‌ی ۳ می‌گیرد).

NTILE(n) ردیف‌ها را تا حد امکان مساوی به n گروه (بازه) تقسیم می‌کند — مثلاً NTILE(4) داده را به چهار چارک (quartile)

تقسیم می‌کند؛ کاربرد رایج آن گزارش‌های تحلیلی مثل «۲۵٪ گران‌ترین محصولات» است.

هشدار: اگر ستون ORDER BY شما مقدار تکراری داشته باشد، ROW_NUMBER می‌تواند در اجراهای مختلف، شماره‌های متفاوتی به ردیف‌های برابر بدهد (چون ترتیب داخلی آن‌ها مشخص نیست). برای نتیجه‌ی همیشه یکسان و قابل پیش‌بینی، یک ستون کاملاً یکتا (مثل کلید اصلی) را هم به ORDER BY اضافه کنید، مثلاً ORDER BY ListPrice DESC, ProductID.

۴.۴.۵ توابع افست (Offset Functions): LAG و LEAD

LAG و LEAD به شما اجازه می‌دهند مقدار یک ستون را از یک ردیف قبلی یا بعدی (نسبت به ردیف فعلی، درون همان پارتیشن) بدون نیاز به SELF JOIN بخوانید.

```
1 SELECT
2     CustomerID ,
3     SalesOrderID ,
4     OrderDate ,
5     TotalDue ,
6     LAG(TotalDue) OVER (PARTITION BY CustomerID ORDER BY OrderDate)
7     AS PreviousOrderAmount ,
8     LEAD(TotalDue) OVER (PARTITION BY CustomerID ORDER BY OrderDate)
9     AS NextOrderAmount
10 FROM Sales.SalesOrderHeader ;
```

این کوئری برای هر سفارش، مبلغ سفارش قبلی و سفارش بعدی همان مشتری را (به ترتیب تاریخ) کنار همان ردیف نشان می‌دهد — بدون اینکه هیچ ردیفی حذف یا خلاصه شود.
نحو کامل و آرگومان‌های اختیاری:

```
1 LAG ( scalar_expression [ , offset ] [ , default ] ) OVER ( ... )
2 LEAD ( scalar_expression [ , offset ] [ , default ] ) OVER ( ... )
```

• offset (پیش فرض ۱): چند ردیف عقب‌تر (برای LAG) یا جلوتر (برای LEAD) برویم.

• default (پیش فرض NULL): چه مقداری برگردانده شود وقتی ردیف قبلی / بعدی وجود ندارد (مثلاً برای اولین سفارش هر مشتری).

```
1 -- go back 2 rows instead of 1, and use 0 instead of NULL when there
  is no previous row
2 SELECT CustomerID, OrderDate, TotalDue,
3        LAG(TotalDue, 2, 0) OVER (PARTITION BY CustomerID ORDER BY
  OrderDate) AS TwoOrdersAgo
4 FROM Sales.SalesOrderHeader;
```

کاربردهای رابج LEAD/LAG: محاسبه‌ی درصد تغییر نسبت به دوره‌ی قبل، شناسایی شکاف‌های زمانی بین رویدادها، مقایسه‌ی مقدار فعلی با مقدار قبلی / بعدی بدون SELF JOIN.

۵.۴.۵ تعریف چارچوب پنجره: RANGE BETWEEN و ROWS BETWEEN

تا اینجا، پنجره‌ی هر محاسبه به طور ضمنی «همه‌ی ردیف‌های همان پارتیشن» بود. اما می‌توانید با عبارت **Window Frame** دقیقاً مشخص کنید محاسبه فقط روی کدام زیرمجموعه از ردیف‌های پارتیشن انجام شود — این دقیقاً همان چیزی است که جمع تجمعی (Run-ning Total) یا میانگین متحرک (Moving Average) را ممکن می‌کند.
نحو کلی:

```
1 ROWS | RANGE BETWEEN <start> AND <end>
```

جایی که <start> و <end> می‌توانند یکی از این‌ها باشند:

• UNBOUNDED PRECEDING — از اولین ردیف پارتیشن

• N – N PRECEDING — N ردیف قبل از ردیف فعلی

• CURRENT ROW — خود ردیف فعلی

• N – N FOLLOWING — N ردیف بعد از ردیف فعلی

• UNBOUNDED FOLLOWING — تا آخرین ردیف پارتیشن

مثال ۱: جمع تجمعی (Running Total)

```
1 SELECT
2     SalesOrderID,
3     OrderDate,
4     TotalDue,
5     SUM(TotalDue) OVER (
6         ORDER BY OrderDate
7         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
8     ) AS RunningTotal
9 FROM Sales.SalesOrderHeader;
```

برای هر سفارش، این کوئری مجموع مبلغ همه‌ی سفارش‌های تا همین سفارش (براساس تاریخ) را نشان می‌دهد — دقیقاً همان چیزی که یک صفحه‌ی گزارش موجودی بانک با ستون «مانده‌ی حساب» نشان می‌دهد.

نکته‌ی کاربردی: اگر بعد از ORDER BY داخل OVER، هیچ Window Frame ننویسید، T-SQL به طور پیش فرض معادل RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW عمل می‌کند — یعنی اگر فقط SUM(...) OVER (ORDER BY OrderDate) بنویسید، همان جمع تجمعی را می‌گیرید، بدون نیاز به نوشتن صریح ROWS BETWEEN.

مثال ۲: میانگین متحرک سه‌روزه (Moving Average)

```
1 SELECT
2     SalesOrderID ,
3     OrderDate ,
4     TotalDue ,
5     AVG(TotalDue) OVER (
6         ORDER BY OrderDate
7         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
8     ) AS MovingAvg3Orders
9 FROM Sales.SalesOrderHeader;
```

این کوئری برای هر سفارش، میانگین مبلغ همان سفارش به همراه دو سفارش قبلی را محاسبه می‌کند — یک پنجره‌ی «لغزنده» با عرض ثابت ۳ ردیف.

تفاوت ROWS و RANGE:

	p۰.۷۵ p۰.۱۵
کلمه تعریف چارچوب بر چه اساسی است	
ROWS	بر اساس تعداد فیزیکی ردیف‌ها (مثلاً «۲ ردیف قبل»)، فارغ از اینکه چند ردیف مقدار ORDER BY یکسانی داشته باشند.
RANGE	بر اساس مقدار منطقی ستون ORDER BY: اگر چند ردیف مقدار ORDER BY دقیقاً یکسانی داشته باشند (مثلاً چند سفارش در یک تاریخ)، RANGE همه‌ی آن‌ها را با هم به عنوان یک واحد در نظر می‌گیرد، نه به صورت جداگانه.

در عمل، برای اکثر گزارش‌های تجمعی (جمع تجمعی، میانگین متحرک)، ROWS انتخاب رایج‌تر و قابل پیش‌بینی‌تری است، مگر اینکه دقیقاً بخواهید ردیف‌های هم‌ارزش را یک واحد حساب کنید.

۶.۴.۵ جمع‌بندی توابع پنجره‌ای

- توابع پنجره‌ای، برخلاف GROUP BY، هیچ ردیفی را خلاصه نمی‌کنند؛ فقط یک ستون محاسبه‌شده به هر ردیف اضافه می‌کنند.
- RANK, ROW_NUMBER, DENSE_RANK و NTILE در مواجهه با مقادیر برابر رفتار متفاوتی دارند — برای انتخاب درست، به نیاز واقعی گزارش دقت کنید.
- LEAD و LAG خواندن مقدار ردیف قبلی/بعدی را بدون SELF JOIN ممکن می‌کنند و آرگومان‌های offset و default دارند.
- ROWS BETWEEN و RANGE BETWEEN چارچوب دقیق محاسبه‌ی توابع تجمعی پنجره‌ای (مثل SUM, AVG) را کنترل می‌کنند و پایه‌ی جمع تجمعی و میانگین متحرک هستند.

۵.۵ توابع تجمعی (Aggregate Functions)

توابع تجمعی مثل SUM, MAX و AVG چند مقدار را ورودی می‌گیرند و یک نتیجه‌ی تکی برمی‌گردانند. مهم‌ترین و پرکاربردترین این توابع عبارت‌اند از SUM, MAX, MIN, AVG و COUNT.

۱.۵.۵ نکات مهم درباره‌ی توابع تجمعی

- توابع تجمعی یک مقدار (اسکالر) تکی برمی‌گردانند و تقریباً هرجایی که یک مقدار تکی مجاز باشد قابل استفاده‌اند — مثلاً در SELECT, HAVING و ORDER BY. اما نمی‌توانند در عبارت WHERE استفاده شوند.
- توابع تجمعی مقادیر NULL را نادیده می‌گیرند، به جز COUNT(*).
- اگر برای یک تابع تجمعی در لیست SELECT نام مستعار (alias) نگذارید، هیچ عنوان ستونی نخواهد داشت.

• اگر عبارت GROUP BY نداشته باشید، تابع تجمیعی روی **همه**ی ردیف‌هایی که از فیلتر WHERE عبور کرده‌اند اعمال می‌شود.

• تا وقتی از GROUP BY استفاده نکرده‌اید، نباید توابع تجمیعی را با ستون‌های عادی (که داخل خود تابع تجمیعی نیستند) در یک لیست SELECT ترکیب کنید.

۲.۵.۵ جدول توابع تجمیعی پرکاربرد

تابع نحو توضیح
SUM(expression) SUM مجموع مقادیر عددی غیر NULL یک ستون.
AVG(expression) AVG میانگین مقادیر عددی غیر NULL (مجموع تقسیم بر تعداد).
MIN(expression) MIN کوچک‌ترین عدد، قدیمی‌ترین تاریخ/زمان، یا اولین رشته (بر اساس ترتیب collation).
MAX(expression) MAX بزرگ‌ترین عدد، جدیدترین تاریخ/زمان، یا آخرین رشته (بر اساس ترتیب collation).
COUNT(expression) یا COUNT(*) COUNT / COUNT_BIG تعداد ردیف‌ها (حتی NULL ها) را می‌شمارد؛ با یک ستون مشخص، فقط تعداد ردیف‌های غیر NULL آن ستون را می‌شمارد.

۳.۵.۵ مثال ساده

```
1 SELECT AVG(ListPrice) AS AveragePrice,
2     MIN(ListPrice) AS MinimumPrice,
3     MAX(ListPrice) AS MaximumPrice
4 FROM Production.Product;
```

می‌توان با اضافه کردن WHERE، این محاسبات را به یک دسته‌بندی خاص محدود کرد:

```
1 SELECT AVG(ListPrice) AS AveragePrice,
2     MIN(ListPrice) AS MinimumPrice,
3     MAX(ListPrice) AS MaximumPrice
4 FROM Production.Product
5 WHERE ProductCategoryID = 15;
```

۴.۵.۵ قانون مهم: هر ستون یا در تابع تجمیعی باشد، یا در GROUP BY

اگر بدون GROUP BY، یک ستون عادی را کنار توابع تجمیعی در SELECT بگذارید، خطا می‌گیرید:

```
1 -- this query fails:
2 SELECT ProductCategoryID, AVG(ListPrice) AS AveragePrice,
3     MIN(ListPrice) AS MinimumPrice,
4     MAX(ListPrice) AS MaximumPrice
5 FROM Production.Product;
```

```
1 Msg 8120, Level 16, State 1, Line 1
2 Column 'Production.ProductCategoryID' is invalid in the select list
3 because it isn't contained in either an aggregate function or the
4   GROUP BY clause.
```

چون بدون GROUP BY، کوئری همه‌ی ردیف‌ها را یک گروه واحد در نظر می‌گیرد؛ پس همه‌ی ستون‌های SELECT باید ورودی یک تابع تجمیعی باشند.

۵.۵.۵ تجمیع روی تاریخ و متن

توابع تجمیعی محدود به داده‌ی عددی نیستند؛ MIN و MAX روی تاریخ و متن هم کار می‌کنند:

```
1 -- first and last company name alphabetically
2 SELECT MIN(CompanyName) AS MinCustomer,
3     MAX(CompanyName) AS MaxCustomer
4 FROM SalesLT.Customer;
```

می‌توان یک تابع اسکالر را داخل یک تابع تجمیعی هم به کار برد؛ مثلاً استخراج فقط سال از تاریخ سفارش قبل از اعمال MAX/MIN:

```
1 SELECT MIN(YEAR(OrderDate)) AS Earliest,
2        MAX(YEAR(OrderDate)) AS Latest
3 FROM Sales.SalesOrderHeader;
```

نکته: AVG و SUM فقط روی داده‌ی عددی (عدد صحیح، decimal، float، money) کار می‌کنند؛ اما MAX/MIN می‌توانند روی تاریخ هم استفاده شوند.

۶.۵.۵ استفاده از DISTINCT داخل تابع تجمیعی

با گذاشتن DISTINCT داخل یک تابع تجمیعی، مقادیر تکراری ستون ورودی قبل از محاسبه حذف می‌شوند. این کار برای شمارش تعداد مقادیر یکتا بسیار مفید است؛ مثلاً شمارش تعداد مشتریانی که حداقل یک سفارش ثبت کرده‌اند (فارغ از اینکه چند بار سفارش داده‌اند):

```
1 SELECT COUNT(DISTINCT CustomerID) AS UniqueCustomers
2 FROM Sales.SalesOrderHeader;
```

COUNT(< >) فقط تعداد ردیف‌هایی که در آن ستون مقدار دارند را می‌شمارد (اگر NULL نداشته باشد، دقیقاً برابر COUNT(*) است). COUNT(DISTINCT < >) تعداد مقادیر متفاوت آن ستون را می‌شمارد.

۷.۵.۵ تعامل توابع تجمیعی با NULL

- به جز COUNT(*)، بقیه‌ی توابع تجمیعی مقادیر NULL را نادیده می‌گیرند. مثلاً SUM فقط مقادیر غیر NULL را جمع می‌زند؛ NULL معادل صفر در نظر گرفته نمی‌شود.

- وجود NULL در یک ستون می‌تواند محاسبه‌ی AVG را از حالتی که انتظار دارید متفاوت کند، چون AVG مجموع مقادیر پر شده را بر تعداد ردیف‌های غیر NULL تقسیم می‌کند، نه بر تعداد کل ردیف‌ها.

مثال روشن‌کننده — جدولی با ۶ ردیف که یکی از مقادیر ستون c2 آن NULL است:

```
1 SELECT SUM(c2) AS sum_nonnulls,
2        COUNT(*) AS count_all_rows,
3        COUNT(c2) AS count_nonnulls,
4        AVG(c2) AS average,
5        (SUM(c2)/COUNT(*)) AS arith_average
6 FROM t1;
```

نتیجه نشان می‌دهد average (که خود AVG محاسبه می‌کند) برابر ۳۰ است (۱۵۰ تقسیم بر ۵ ردیف غیر NULL)، در حالی که arith_average (که با تقسیم دستی COUNT(*)/SUM محاسبه شده) برابر ۲۵ است (۱۵۰ تقسیم بر کل ۶ ردیف). اگر می‌خواهید همه‌ی ردیف‌ها — چه NULL و چه غیر NULL — در محاسبه لحاظ شوند، می‌توانید با تابع COALESCE مقدار NULL را با یک عدد جایگزین کنید.

۶.۵ گروه‌بندی داده با GROUP BY

توابع تجمیعی برای تحلیل مفیدند، اما گاهی می‌خواهیم قبل از خلاصه‌سازی، داده را به زیرمجموعه‌های مشخص تقسیم کنیم. این کار با عبارت GROUP BY انجام می‌شود.

۱.۶.۵ نحو کلی

```
1 GROUP BY <value1> [, <value2>, ...]
```

GROUP BY ردیف‌های جدول مجازی حاصل از FROM و WHERE را بر اساس مقادیر مشخص شده به گروه‌هایی تقسیم می‌کند.

```

1 SELECT CustomerID
2 FROM Sales.SalesOrderHeader
3 GROUP BY CustomerID;

```

این کوئری معادل زیر است:

```

1 SELECT DISTINCT CustomerID
2 FROM Sales.SalesOrderHeader;

```

اما فرق اصلی GROUP BY با DISTINCT اینجاست: با GROUP BY می‌توانیم عناصر دیگری هم به لیست SELECT اضافه کنیم که برای هر گروه **تجميع** می‌شوند. مثال:

```

1 SELECT CustomerID, COUNT(*) AS OrderCount
2 FROM Sales.SalesOrderHeader
3 GROUP BY CustomerID;

```

این کوئری برای هر CustomerID، تعداد سفارش‌های آن مشتری را می‌شمارد و در یک ردیف خلاصه می‌کند.

۳.۶.۵ ترتیب پردازش و رابطه با ORDER BY

GROUP BY ترتیب خاصی برای نتایج تضمین نمی‌کند. گاهی به‌طور طبیعی نتیجه بر اساس ترتیب مقادیر گروه‌بندی برمی‌گردد، اما نباید روی این رفتار حساب باز کنید. اگر ترتیب مشخصی می‌خواهید، حتماً ORDER BY را صریح اضافه کنید:

```

1 SELECT CustomerID, COUNT(*) AS OrderCount
2 FROM Sales.SalesOrderHeader
3 GROUP BY CustomerID
4 ORDER BY CustomerID;

```

یادآوری از ترتیب پردازش منطقی یک SELECT:

```

1 FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY

```

چون alias ها در مرحله‌ی SELECT تعیین می‌شوند (که بعد از GROUP BY ولی قبل از ORDER BY است)، می‌توانید از alias در ORDER BY استفاده کنید، اما **نمی‌توانید** آن را در GROUP BY به‌کار ببرید. برای مثال، کوئری زیر خطا می‌دهد:

```

1 -- error: cannot use an alias in GROUP BY
2 SELECT CustomerID AS Customer,
3        COUNT(*) AS OrderCount
4 FROM Sales.SalesOrderHeader
5 GROUP BY Customer
6 ORDER BY Customer;

```

اما این کوئری درست کار می‌کند، چون در GROUP BY از نام واقعی ستون استفاده شده:

```

1 SELECT CustomerID AS Customer,
2        COUNT(*) AS OrderCount
3 FROM Sales.SalesOrderHeader
4 GROUP BY CustomerID
5 ORDER BY Customer;

```

۴.۶.۵ رفع خطای رایج GROUP BY

خطای زیر یکی از رایج‌ترین چالش‌های تازه‌کارهای T-SQL است:

```

1 Msg 8120, Level 16, State 1, Line 2
2 Column <column_name> is invalid in the select list because it is not
3 contained in either an aggregate function or the GROUP BY clause.

```

مثال از کوئری ای که این خطا را می دهد:

```
1 -- error: PurchaseOrderNumber is neither in GROUP BY nor in an
  aggregate function
2 SELECT CustomerID, PurchaseOrderNumber, COUNT(*) AS OrderCount
3 FROM Sales.SalesOrderHeader
4 GROUP BY CustomerID;
```

دلیل خطا: این کوئری برای هر CustomerID فقط یک ردیف برمی گرداند. اما ممکن است هر مشتری چند PurchaseOrderNumber مختلف داشته باشد — پس Server SQL نمی داند کدام مقدار PurchaseOrderNumber را باید نشان دهد.

راه حل: اگر می خواهید سفارش ها را هم بر اساس مشتری و هم بر اساس شماری سفارش خرید گروه بندی کنید، آن ستون را هم به GROUP BY اضافه کنید:

```
1 SELECT CustomerID, PurchaseOrderNumber, COUNT(*) AS OrderCount
2 FROM Sales.SalesOrderHeader
3 GROUP BY CustomerID, PurchaseOrderNumber;
```

این کوئری برای هر ترکیب مشتری و شماری سفارش خرید، یک ردیف همراه با تعداد سفارش های آن ترکیب برمی گرداند.

۷.۵ فیلتر کردن گروه ها با HAVING

بعد از اینکه با GROUP BY گروه ها را ساختید، می توانید نتایج را بیشتر فیلتر کنید. عبارت HAVING نقش یک فیلتر روی گروه ها دارد؛ درست همان طور که WHERE روی ردیف های خروجی FROM فیلتر می کند.

۱.۷.۵ نحو و مثال

```
1 SELECT CustomerID,
2        COUNT(*) AS OrderCount
3 FROM Sales.SalesOrderHeader
4 GROUP BY CustomerID
5 HAVING COUNT(*) > 10;
```

این کوئری تعداد سفارش های هر مشتری را می شمارد و فقط مشتریانی که بیش از ۱۰ سفارش ثبت کرده اند را در نتیجه نگه می دارد.

۲.۷.۵ تفاوت HAVING و WHERE

HAVING	WHERE	
بعد از تشکیل گروه ها	قبل از تشکیل گروه ها	زمان اجرا
کل یک گروه (معمولاً بر اساس نتیجه ی یک تابع تجمیعی)	تک تک ردیف های خام	روی چه چیزی فیلتر می کند

اگر کوئری شما هم WHERE و هم GROUP BY ... HAVING داشته باشد، ابتدا WHERE ردیف ها را فیلتر می کند و سپس GROUP BY روی ردیف های باقی مانده گروه بندی انجام می دهد؛ یعنی WHERE می تواند از قبل، محدوده ی گروه هایی که اصلاً ساخته می شوند را کوچک تر کند.

خلاصه:

• WHERE ردیف ها را قبل از تشکیل گروه ها فیلتر می کند.

• HAVING کل یک گروه را فیلتر می کند و معمولاً به نتیجه ی یک تجمیع نگاه می کند.

۸.۵ جمع‌بندی و نکات طلایی

- توابع T-SQL به پنج دسته تقسیم می‌شوند: **Aggregate** و **Rowset**, **Ranking**, **Logical**, **Scalar**.
- توابع **Scalar** روی هر ردیف، یک مقدار تکی برمی‌گردانند (مثل **GETDATE**, **ROUND**, **UPPER**). به قطعی / غیرقطعی بودن آن‌ها توجه کنید.
- توابع **Ranking** (مثل **RANK**) با عبارت **OVER** و در صورت نیاز **PARTITION BY** روی مجموعه‌ای از ردیف‌ها کار می‌کنند.
- توابع **Rowset** (مثل **OPENJSON**, **OPENROWSET**) یک جدول مجازی برمی‌گردانند که در **FROM** قابل استفاده است.
- توابع **Aggregate** (مثل **COUNT**, **MAX**, **MIN**, **AVG**, **SUM**) چند مقدار را به یک مقدار خلاصه تبدیل می‌کنند؛ در **WHERE** قابل استفاده نیستند و (به جز **COUNT(*)**) مقادیر **NULL** را نادیده می‌گیرند.
- بدون **GROUP BY**، هر ستون عادی در کنار یک تابع تجمیعی خطا می‌دهد — یا ستون باید ورودی یک تابع تجمیعی باشد، یا داخل **GROUP BY** بیاید.
- **GROUP BY** داده را به زیرگروه‌ها تقسیم می‌کند تا بتوان روی هر گروه، به‌طور جداگانه تجمیع کرد.
- **HAVING** گروه‌ها را فیلتر می‌کند (بعد از **GROUP BY**), در حالی که **WHERE** ردیف‌ها را قبل از گروه‌بندی فیلتر می‌کند.
- ترتیب پردازش کامل یک **SELECT**: **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **ORDER BY**.

منبع

این نوشته بر اساس مطالب مائول رسمی مایکروسافت لرن تهیه شده است: [Transact-SQL in BY GROUP and functions built-in Use](#)
[Learn Microsoft](#)

۶ تغییر داده (INSERT/UPDATE/DELETE/MERGE)

این فصل خلاصه‌ای ساده‌شده و به زبان فارسی از مائول ششم و آخر مسیر یادگیری مایکروسافت Query and modify data with Transact-SQL است؛ مائول Modify data with T-SQL که یاد می‌دهد چطور با **UPDATE**, **INSERT**, **DELETE** و **MERGE** داده را در جدول‌ها اضافه، تغییر یا حذف کنیم.

۱.۶ درج داده با INSERT

دستور **INSERT** برای اضافه کردن یک یا چند ردیف به یک جدول استفاده می‌شود و چند شکل مختلف دارد.

۱.۱.۶ نحوه پایه (VALUES) INSERT

```
1 INSERT [INTO] <Table> [(column_list)]  
2 VALUES ([ColumnName or an expression or DEFAULT or NULL], ...n)
```

با این فرم — که به آن **VALUES INSERT** گفته می‌شود — می‌توانید مشخص کنید داده در کدام ستون‌ها و با چه ترتیبی درج شود. نوشتن لیست ستون‌ها (**column_list**) اختیاری است، اما توصیه می‌شود؛ اگر آن را ننویسید، T-SQL انتظار دارد برای **همه‌ی** ستون‌های جدول، به همان ترتیبی که هنگام ساخت جدول تعریف شده‌اند، مقدار بدهید.

۲.۱.۶ کلمه‌ی کلیدی DEFAULT

وقتی به جای یک مقدار، کلمه‌ی **DEFAULT** را می‌نویسید، یکی از سه حالت زیر رخ می‌دهد:

- اگر ستون یک مقدار خودکار (**Auto-generated**) داشته باشد (مثل **IDENTITY**) همان مقدار استفاده می‌شود.
- اگر هنگام ساخت جدول یک مقدار پیش‌فرض برای آن ستون تعریف شده باشد، همان مقدار استفاده می‌شود.
- اگر ستون اجازه‌ی **NULL** داشته باشد و نه مقدار خودکار دارد و نه مقدار پیش‌فرض، مقدار **NULL** درج می‌شود.

نکته کاربردی: برای دیدن سریع نام ستون‌های یک جدول (بدون واکنشی هیچ ردیفی)، می‌توانید یک شرط همیشه نادرست بنویسید:

```
1 SELECT * FROM Sales.Promotion
2 WHERE 1 = 0;
```

۳.۱.۶ مثال‌های INSERT

اگر برای همه‌ی ستون‌ها و به ترتیب درست مقدار بدهید، نوشتن column_list اختیاری است:

```
1 INSERT INTO Sales.Promotion (PromotionName, StartDate, ProductModelID
2   , Discount, Notes)
3 VALUES
4 ('Clearance Sale', '01/01/2021', 23, 0.1, '10% discount');
5 -- equivalent without column_list, since the number and order of
6   values matches the columns:
7 INSERT INTO Sales.Promotion
8 VALUES
9 ('Clearance Sale', '01/01/2021', 23, 0.1, '10% discount');
```

استفاده‌ی صریح از NULL و DEFAULT:

```
1 INSERT INTO Sales.Promotion
2 VALUES
3 ('Pull your socks up', DEFAULT, 24, 0.25, NULL);
```

اگر نمی‌خواهید برای همه‌ی ستون‌ها مقدار بدهید، باید حتماً column_list را بنویسید تا مشخص شود مقادیرتان برای کدام ستون‌هاست؛ بقیه‌ی ستون‌ها یا مقدار پیش‌فرض می‌گیرند یا (در صورت مجاز بودن) NULL:

```
1 INSERT INTO Sales.Promotion (PromotionName, ProductModelID, Discount)
2 VALUES
3 ('Caps Locked', 2, 0.2);
```

۴.۱.۶ درج چند ردیف همزمان (Table Value Constructor)

می‌توانید با نوشتن چند مجموعه مقدار، جداشده با کاما، چند ردیف را در یک دستور INSERT درج کنید. به این ساختار Value Table Constructor گفته می‌شود:

```
1 INSERT INTO Sales.Promotion
2 VALUES
3 ('The gloves are off!', DEFAULT, 3, 0.25, NULL),
4 ('The gloves are off!', DEFAULT, 4, 0.25, NULL);
```

۲.۶ INTO ... SELECT و SELECT ... INSERT

۱.۲.۶ SELECT ... INSERT

به جای نوشتن مقادیر ثابت، می‌توانید نتیجه‌ی یک SELECT دیگر (یا خروجی یک Stored Procedure) را به عنوان منبع داده‌ی INSERT استفاده کنید. به این فرم INSERT SELECT گفته می‌شود:

```
1 INSERT [INTO] <table or view> [(column_list)]
2 SELECT <column_list> FROM <table_list>...;
```

مثال: درج چند سفارش تبلیغاتی جدید با نام «Get» «Framed» بر اساس مدل‌هایی که در نامشان کلمه‌ی «frame» وجود دارد:

```

1 INSERT INTO Sales.Promotion (PromotionName, ProductModelID, Discount,
   Notes)
2 SELECT DISTINCT 'Get Framed', m.ProductModelID, 0.1, '10% off ' + m.
   Name
3 FROM Production.ProductModel AS m
4 WHERE m.Name LIKE '%frame%';

```

نکته: برخلاف Subquery ها، کوئری SELECT تودرتوی استفاده شده در INSERT SELECT نیازی به پرانتز ندارد.

فرم دیگری هم وجود دارد به نام **INSERT EXEC** که خروجی یک Stored Procedure را ورودی INSERT می کند؛ از نظر مفهومی مشابه INSERT SELECT است، اما چون یک Procedure می تواند چند نتیجه (result set) برگرداند، باید با احتیاط بیشتری استفاده شود.

۲.۲.۶ SELECT ... INTO

گزینه‌ی دیگری برای درج ردیف‌ها، دستور SELECT INTO است. تفاوت اصلی‌اش با INSERT SELECT این است: SE-LECT INTO نمی‌تواند به یک جدول موجود داده درج کند، چون همیشه یک جدول جدید بر اساس نتیجه‌ی SELECT می‌سازد. هر ستون جدول جدید، همان نام، نوع داده و خاصیت nullable بودن ستون (یا عبارت) متناظر در لیست SELECT را می‌گیرد. برای استفاده از این دستور، کافی است `INTO < __ >` را در عبارت SELECT درست قبل از FROM اضافه کنید:

```

1 SELECT SalesOrderID, CustomerID, OrderDate, PurchaseOrderNumber,
   TotalDue
2 INTO Sales.Invoice
3 FROM Sales.SalesOrderHeader;

```

اگر از قبل جدولی با همین نام (بعد از INTO) وجود داشته باشد، این دستور با خطا مواجه می‌شود. بعد از ساخته شدن، این جدول جدید مثل هر جدول دیگری قابل استفاده است — می‌توان از آن SELECT گرفت، آن را JOIN کرد یا داده‌ی بیشتری در آن درج کرد.

۳.۶ تولید مقادیر خودکار: IDENTITY

گاهی نیاز داریم مقادیر یک ستون به‌طور خودکار و متوالی تولید شوند (مثلاً شماره‌ی سریالی برای هر ردیف جدید). T-SQL دوره برای این کار دارد: خاصیت IDENTITY روی یک ستون مشخص، یا ساختن یک آبجکت SEQUENCE مستقل.

۱.۳.۶ تعریف ستون IDENTITY

برای استفاده از IDENTITY، یک ستون با نوع داده‌ی عددی و مقیاس صفر (یعنی فقط عدد صحیح) تعریف کرده و کلمه‌ی کلیدی IDENTITY را به آن اضافه کنید. یک مقدار شروع (seed) و یک گام (increment) اختیاری هم می‌توان مشخص کرد؛ اگر ننویسید، هر دو برابر با در نظر گرفته می‌شوند.

```

1 CREATE TABLE Sales.Promotion
2 (
3 PromotionID int IDENTITY PRIMARY KEY,
4 PromotionName varchar(20),
5 StartDate datetime NOT NULL DEFAULT GETDATE(),
6 ProductModelID int NOT NULL REFERENCES Production.ProductModel(
   ProductModelID),
7 Discount decimal(4,2) NOT NULL,
8 Notes nvarchar(max) NULL
9 );

```

نکات مهم:

- در یک جدول، فقط یک ستون می‌تواند خاصیت IDENTITY داشته باشد؛ معمولاً همان کلید اصلی یا یک کلید جایگزین است.
- ستون IDENTITY به‌طور خودکار NOT NULL است.

۲.۳.۶ درج داده در ستون IDENTITY

وقتی یک ستون IDENTITY دارید، معمولاً هنگام INSERT مقداری برای آن ستون مشخص نمی‌کنید — دیتابیس خودش بعدی‌ترین مقدار موجود را تولید می‌کند:

```
1 INSERT INTO Sales.Promotion
2 VALUES
3 ('Clearance Sale', '01/01/2021', 23, 0.10, '10% discount');
```

نکته‌ی جالب: با اینکه VALUES مقداری برای PromotionID ندارد، نیازی نیست column_list را بنویسید — ستون‌های IDENTITY از این قاعده مستثنا هستند.

۳.۳.۶ گرفتن آخرین مقدار IDENTITY

- برای گرفتن آخرین مقدار IDENTITY تولیدشده در همین session و scope:

```
1 SELECT SCOPE_IDENTITY();
```

- برای گرفتن آخرین مقدار IDENTITY در یک جدول مشخص (فارغ از session):

```
1 SELECT IDENT_CURRENT('Sales.Promotion');
```

۴.۳.۶ بازنویسی مقدار IDENTITY

اگر بخواهید به جای مقدار خودکار، یک مقدار مشخص وارد کنید، ابتدا باید حالت درج دستی را فعال کنید:

```
1 SET IDENTITY_INSERT SalesLT.Promotion ON;
2
3 INSERT INTO SalesLT.Promotion (PromotionID, PromotionName,
4   ProductModelID, Discount)
5 VALUES
6 (20, 'Another short sale', 37, 0.3);
7 SET IDENTITY_INSERT SalesLT.Promotion OFF;
```

۵.۳.۶ ریست کردن IDENTITY

گاهی لازم است مقدار IDENTITY را بازنشانی یا رد کنید (مثلاً بعد از حذف همه‌ی ردیف‌های جدول، مقدار بعدی را دوباره از ۱ شروع کنید). این کار با تابع DBCC CHECKIDENT انجام می‌شود.

۴.۶ تولید مقادیر خودکار: SEQUENCE

SEQUENCE یک آبجکت مستقل از هر جدول است که می‌تواند مقادیر متوالی تولید کند. با دستور CREATE SEQUENCE ساخته می‌شود:

```
1 CREATE SEQUENCE Sales.InvoiceNumber AS INT
2 START WITH 1000 INCREMENT BY 1;
```

برای گرفتن مقدار بعدی، از NEXT VALUE FOR استفاده کنید:

```
1 INSERT INTO Sales.ResellerInvoice
2 VALUES
3 (NEXT VALUE FOR Sales.InvoiceNumber, 2, GETDATE(), 'P012345', 107.99)
4 ;
```

۱.۴.۶ چه زمانی SEQUENCE بهتر از IDENTITY است؟

- وقتی برنامه‌ی شما نیاز دارد یک سری عددی مشترک را بین چند جدول یا چند ستون به اشتراک بگذارد — چیزی که IDENTITY نمی‌تواند تضمین کند.
- وقتی می‌خواهید مقادیر را بر اساس یک ستون دیگر مرتب کنید. NEXT VALUE FOR می‌تواند همراه OVER استفاده شود تا مقادیر دقیقاً به ترتیب ORDER BY تولید شوند:

```
1 SELECT NEXT VALUE FOR dbo.Sequence OVER (ORDER BY Name) AS NextID,
2     ProductID,
3     Name
4 FROM Production.Product;
```

نکته: حتی اگر فقط برای نمایش این مقادیر را SELECT بگیرید، همچنان «مصرف» می‌شوند و دیگر قابل استفاده مجدد نیستند. هر بار که این کوئری را اجرا کنید، مقادیر متفاوتی می‌گیرید.

- وقتی نیاز دارید چند عدد را همزمان رزرو کنید (مثلاً یک اپلیکیشن می‌خواهد ۵ عدد متوالی را یک‌جا بگیرد). با IDENTITY، اگر چند فرآیند همزمان درخواست بدهند، ممکن است در توالی شکاف بیفتد. برای این منظور می‌توان از Procedure Stored سیستمی sp_sequence_get_range استفاده کرد.
- SEQUENCE امکان تغییر مشخصات آن (مثل مقدار گام) را هم می‌دهد.
- برخلاف SEQUENCE، مقادیر IDENTITY در برابر UPDATE محافظت شده‌اند؛ اگر بخواهید مستقیماً یک ستون IDENTITY را آپدیت کنید، خطا می‌گیرید.

۵.۶ به‌روزرسانی داده با UPDATE

دستور UPDATE برای تغییر داده‌ی موجود در یک جدول استفاده می‌شود. UPDATE روی مجموعه‌ای از ردیف‌ها عمل می‌کند که یا با شرط WHERE مشخص شده‌اند، یا از طریق یک JOIN تعیین می‌شوند.

۱.۵.۶ نحو پایه

```
1 UPDATE <TableName>
2 SET
3 <ColumnName> = { expression | DEFAULT | NULL }
4 {,...n}
5 WHERE <search_conditions>;
```

هشدار مهم: اگر UPDATE را بدون WHERE (یا بدون JOIN) بنویسید، همه‌ی ردیف‌های جدول به‌روزرسانی می‌شوند! همیشه با احتیاط استفاده کنید.

۲.۵.۶ مثال ساده

```
1 UPDATE Sales.Promotion
2 SET Notes = '25% off socks'
3 WHERE PromotionID = 2;
```

۳.۵.۶ تغییر چند ستون همزمان

```
1 UPDATE Sales.Promotion
2 SET Discount = 0.2, Notes = REPLACE(Notes, '10%', '20%')
3 WHERE PromotionName = 'Get Framed';
```

۴.۵.۶ UPDATE با FROM (به روزرسانی بر اساس نتیجه‌ی یک کوئری)

UPDATE هم می‌تواند شامل عبارت FROM باشد تا بتوانید داده را بر اساس نتیجه‌ی یک کوئری دیگر تغییر دهید:

```
1 UPDATE Sales.Promotion
2 SET Notes = FORMAT(Discount, 'P') + ' off ' + m.Name
3 FROM Product.ProductModel AS m
4 WHERE Notes IS NULL
5 AND Sales.Promotion.ProductModelID = m.ProductModelID;
```

۶.۶ حذف داده با DELETE و TABLE TRUNCATE

همان‌طور که INSERT همیشه کل یک ردیف را اضافه می‌کند، DELETE هم همیشه کل یک ردیف را حذف می‌کند (نه بخشی از یک ردیف).

۱.۶.۶ DELETE برای حذف ردیف‌های مشخص

DELETE هم روی مجموعه‌ای از ردیف‌ها عمل می‌کند که با WHERE یا JOIN مشخص می‌شوند. ساختار WHERE در DELETE دقیقاً مثل WHERE در SELECT است.

```
1 DELETE [FROM] <TableName>
2 WHERE <search_conditions>;
```

هشدار مهم: درست مثل UPDATE، اگر DELETE بدون WHERE نوشته شود، همه‌ی ردیف‌های جدول حذف خواهند شد. با احتیاط استفاده کنید.

مثال: حذف همه‌ی محصولات که دیگر تولید نمی‌شوند (ستون discontinued برابر ۱ است):

```
1 DELETE FROM Production.Product
2 WHERE discontinued = 1;
```

۲.۶.۶ TABLE TRUNCATE برای حذف همه‌ی ردیف‌ها

اگر واقعاً می‌خواهید همه‌ی ردیف‌های یک جدول را حذف کنید و جدول را خالی نگه دارید، به جای DELETE بدون شرط، از TRUNCATE TABLE استفاده کنید. این دستور شرط WHERE را قبول نمی‌کند و همیشه همه‌ی ردیف‌ها را در یک عملیات حذف می‌کند:

```
1 TRUNCATE TABLE Sales.Sample;
```

TRUNCATE TABLE از نظر کارایی، برای حذف کامل ردیف‌های یک جدول، بهینه‌تر از DELETE است.

۷.۶ ترکیب چند عملیات با MERGE

گاهی لازم است دو جدول را با هم همگام‌سازی (Synchronize) کنیم؛ یعنی بر اساس تفاوت میان آن‌ها، در یک جدول ردیف اضافه، به‌روزرسانی یا حذف کنیم. برای این کار، T-SQL دستور MERGE را دارد.

در اصطلاح MERGE:

- جدولی که تغییر می‌کند، جدول هدف (Target) نامیده می‌شود.
- جدولی که برای تعیین تغییرات استفاده می‌شود، جدول منبع (Source) نامیده می‌شود.
- MERGE بر اساس یک یا چند شرط، داده را تغییر می‌دهد:
- اگر داده‌ی منبع در جدول هدف مطابقتی داشته باشد → می‌تواند داده‌ی هدف را به‌روزرسانی کند.
- اگر داده‌ی منبع در جدول هدف مطابقتی نداشته باشد → می‌تواند آن را در هدف درج کند.
- اگر داده‌ی هدف در منبع مطابقتی نداشته باشد → می‌تواند آن ردیف را از هدف حذف کند.

```

1 MERGE INTO schema_name.table_name AS TargetTbl
2 USING (SELECT <select_list>) AS SourceTbl
3 ON (TargetTbl.col1 = SourceTbl.col1)
4 WHEN MATCHED THEN
5     UPDATE SET TargetTbl.col2 = SourceTbl.col2
6 WHEN NOT MATCHED [BY TARGET] THEN
7     INSERT (<column_list>)
8     VALUES (<value_list>)
9 WHEN NOT MATCHED BY SOURCE THEN
10    DELETE;

```

نیازی نیست از همه‌ی این بخش‌ها استفاده کنید — فقط همان‌هایی که به آن‌ها نیاز دارید را بنویسید.

۲.۷.۶ مثال کاربردی

فرض کنید یک جدول موقت (Sales.InvoiceStaging) دارید که ترکیبی از فاکتورهای جدید و به‌روزرسانی فاکتورهای موجود است. با MERGE می‌توانید در یک دستور، هم فاکتورهای موجود را به‌روز کنید، هم فاکتورهای جدید را درج کنید:

```

1 MERGE INTO Sales.Invoice as i
2 USING Sales.InvoiceStaging as s
3 ON i.SalesOrderID = s.SalesOrderID
4 WHEN MATCHED THEN
5     UPDATE SET i.CustomerID = s.CustomerID ,
6                i.OrderDate = GETDATE() ,
7                i.PurchaseOrderNumber = s.PurchaseOrderNumber ,
8                i.TotalDue = s.TotalDue
9 WHEN NOT MATCHED THEN
10    INSERT (SalesOrderID, CustomerID, OrderDate, PurchaseOrderNumber ,
11           TotalDue)
12    VALUES (s.SalesOrderID, s.CustomerID, s.OrderDate, s.
13           PurchaseOrderNumber , s.TotalDue);

```

۳.۷.۶ نکته‌ی مهم درباره‌ی کارایی

MERGE از مکانیزم‌های قفل‌گذاری (Locking) متفاوتی نسبت به دستورات جدای UPDATE, INSERT و DELETE استفاده می‌کند. در مقیاس بزرگ، MERGE می‌تواند مسائل پیچیده‌ای در همزمانی (Concurrency) ایجاد کند؛ پس همیشه هر دستور MERGE را قبل از استفاده در محیط Production به‌طور کامل تست کنید. اگر فقط قرار است یک جدول را براساس ردیف‌های جدول دیگر به‌روزرسانی کنید، نوشتن دستورات جداگانه‌ی UPDATE, INSERT و DELETE معمولاً کارایی و مقیاس‌پذیری بهتری دارد.

۸.۶ جمع‌بندی و نکات طلایی

- INSERT برای اضافه کردن ردیف استفاده می‌شود؛ سه فرم دارد: VALUES INSERT (مقادیر ثابت)، SELECT INSERT (نتیجه‌ی یک کوئری) و INTO SELECT (که یک جدول جدید می‌سازد).
- کلمه‌ی DEFAULT باعث می‌شود مقدار پیش فرض تعریف شده در جدول (یا مقدار خودکار، یا NULL استفاده شود).
- برای درج چند ردیف در یک دستور، از Constructor Value Table (چند مجموعه مقدار جدا شده با کاما) استفاده کنید.
- برای تولید خودکار مقادیر متوالی، از IDENTITY (مخصوص یک ستون در یک جدول) یا SEQUENCE (مستقل و قابل اشتراک بین چند جدول / ستون) استفاده کنید.
- UPDATE روی مجموعه‌ای از ردیف‌ها عمل می‌کند و همیشه باید با WHERE (یا JOIN همراه باشد) — در غیر این صورت همه‌ی ردیف‌ها تغییر می‌کنند.

• **DELETE** هم برای حذف ردیف‌های مشخص باید همراه **WHERE** باشد؛ برای حذف **همه‌ی** ردیف‌ها، **TRUNCATE TABLE** سریع‌تر و بهینه‌تر است.

• **MERGE** به شما اجازه می‌دهد در یک دستور، بین دو جدول (منبع و هدف) همگام‌سازی کنید و بسته به شرط تطابق، **INSERT**، **UPDATE** یا **DELETE** انجام دهید — اما همیشه پیش از استفاده در **Production** آن را کامل تست کنید.

منبع

این نوشته بر اساس مطالب ماژول رسمی مایکروسافت لرن تهیه شده است: [Learn Microsoft — T-SQL with data Modify](#)

۷ رویه‌های ذخیره‌شده (Stored Procedures) در T-SQL

این فصل، برخلاف شش فصل قبلی، مستقیماً از یک ماژول واحد مایکروسافت لرن گرفته نشده است؛ بلکه محتوایی تکمیلی است که مفهوم **Stored Procedure** را — که در فصل‌های اول و ششم فقط به آن اشاره شد — به‌طور مستقل و کامل آموزش می‌دهد. این موضوع معمولاً در مسیرهای پیشرفته‌تر مایکروسافت لرن (مثل **Program with Transact-SQL**) پوشش داده می‌شود.

۱.۷ Stored Procedure چیست و چرا از آن استفاده کنیم؟

Stored Procedure (رویه‌ی ذخیره‌شده) مجموعه‌ای از یک یا چند دستور **T-SQL** است که با یک نام مشخص، داخل خود دیتابیس ذخیره می‌شود و می‌توان آن را بارها با یک دستور ساده (**EXEC**) فراخوانی کرد — دقیقاً مثل یک تابع یا متد در زبان‌های برنامه‌نویسی. مزایای اصلی استفاده از **Stored Procedure**:

- **کارایی بهتر:** **SQL Server** یک نقشه‌ی اجرا (**execution plan**) برای رویه می‌سازد و آن را برای اجراهای بعدی **کش (cache)** می‌کند، پس نیازی به کامپایل مجدد کوئری در هر بار اجرا نیست.
- **امنیت بهتر:** می‌توان به کاربران فقط اجازه‌ی اجرای رویه را داد، بدون اینکه مستقیماً به جدول‌های زیرین دسترسی داشته باشند — این از حملات **SQL Injection** هم تا حد زیادی جلوگیری می‌کند.
- **استفاده‌ی مجدد از کد:** منطق پیچیده‌ی کسب‌وکار یک‌بار نوشته و در یک‌جا نگهداری می‌شود، به جای تکرار همان کوئری در چندین برنامه‌ی مختلف.
- **کاهش ترافیک شبکه:** به جای فرستادن کل متن یک کوئری پیچیده از برنامه به دیتابیس، فقط نام رویه و چند پارامتر کوچک فرستاده می‌شود.

۲.۷ ساخت یک Stored Procedure ساده: CREATE PROCEDURE

```
1 CREATE PROCEDURE Sales.GetCustomerOrders
2 AS
3 BEGIN
4     SELECT SalesOrderID, OrderDate, TotalDue
5     FROM Sales.SalesOrderHeader
6     ORDER BY OrderDate DESC;
7 END;
```

• **CREATE PROCEDURE** (یا مخفف آن **CREATE PROC**) رویه را می‌سازد.

• **BEGIN...END** بدنه‌ی رویه را مشخص می‌کند؛ برای رویه‌های تک‌دستوری اختیاری است، اما برای خوانایی و رویه‌های چنددستوری همیشه توصیه می‌شود.

• نام رویه معمولاً با یک اسکیمای همراه است (مثل **Sales.GetCustomerOrders**), دقیقاً مثل نام جدول‌ها.

۳.۷ پارامترهای ورودی (Input Parameters)

برای اینکه یک رویه بتواند با ورودی‌های متفاوت رفتار متفاوتی داشته باشد، از پارامتر استفاده می‌کنیم:

```
1 CREATE PROCEDURE Sales.GetOrdersByCustomer
2     @CustomerID INT
3 AS
4 BEGIN
5     SELECT SalesOrderID, OrderDate, TotalDue
6     FROM Sales.SalesOrderHeader
7     WHERE CustomerID = @CustomerID
8     ORDER BY OrderDate DESC;
9 END;
```

نام هر پارامتر همیشه با علامت @ شروع می‌شود. برای اجرای این رویه:

```
1 EXEC Sales.GetOrdersByCustomer @CustomerID = 42;
2
3 -- or more simply, by position:
4 EXEC Sales.GetOrdersByCustomer 42;
```

پارامتر با مقدار پیش‌فرض: می‌توانید برای پارامتر یک مقدار پیش‌فرض تعریف کنید تا اجرای رویه بدون فرستادن آن پارامتر هم ممکن باشد:

```
1 CREATE PROCEDURE Sales.GetRecentOrders
2     @DaysBack INT = 30
3 AS
4 BEGIN
5     SELECT SalesOrderID, OrderDate, TotalDue
6     FROM Sales.SalesOrderHeader
7     WHERE OrderDate >= DATEADD(day, -@DaysBack, GETDATE());
8 END;
```

```
1 EXEC Sales.GetRecentOrders;           -- uses the default: last 30 days
2 EXEC Sales.GetRecentOrders 7;         -- overrides it: last 7 days
```

چند پارامتر همزمان:

```
1 CREATE PROCEDURE Sales.AddPromotion
2     @PromotionName VARCHAR(50),
3     @ProductModelID INT,
4     @Discount DECIMAL(4,2)
5 AS
6 BEGIN
7     INSERT INTO Sales.Promotion (PromotionName, ProductModelID,
8     Discount)
9     VALUES (@PromotionName, @ProductModelID, @Discount);
10 END;
```

۴.۷ پارامترهای خروجی (OUTPUT Parameters)

گاهی می‌خواهید یک رویه، علاوه بر انجام کاری، یک مقدار را هم به فراخواننده برگرداند — برای این کار از کلمه‌ی کلیدی OUTPUT استفاده می‌کنیم:

```
1 CREATE PROCEDURE Sales.AddPromotionAndGetID
2     @PromotionName VARCHAR(50),
3     @ProductModelID INT,
```

```

4      @Discount DECIMAL(4,2),
5      @NewPromotionID INT OUTPUT
6 AS
7 BEGIN
8     INSERT INTO Sales.Promotion (PromotionName, ProductModelID,
9     Discount)
10    VALUES (@PromotionName, @ProductModelID, @Discount);
11
12    SET @NewPromotionID = SCOPE_IDENTITY();
13 END;

```

فراخوانی و خواندن مقدار خروجی:

```

1 DECLARE @NewID INT;
2
3 EXEC Sales.AddPromotionAndGetID
4     @PromotionName = 'Spring Sale',
5     @ProductModelID = 12,
6     @Discount = 0.15,
7     @NewPromotionID = @NewID OUTPUT;
8
9 SELECT @NewID AS NewPromotionID;

```

نکته: کلمه‌ی OUTPUT باید هم در تعریف پارامتر داخل CREATE PROCEDURE بیاید، هم هنگام فراخوانی رویه با EXEC. فراموش کردن OUTPUT هنگام فراخوانی باعث می‌شود مقدار واقعی به متغیر شما بازگردانده نشود.

۵.۷ اجرای Stored Procedure با EXEC

```

1 EXEC <procedure_name> [ <parameter_list> ];
2 -- or the full keyword:
3 EXECUTE <procedure_name> [ <parameter_list> ];

```

پارامترها را می‌توان به دوروش فرستاد:

- براساس نام (توصیه شده): EXEC Sales.GetOrdersByCustomer @CustomerID = 42; – خواناتر است و به ترتیب پارامترها وابسته نیست.
- براساس موقعیت: EXEC Sales.GetOrdersByCustomer 42; – باید دقیقاً به همان ترتیبی باشد که در CREATE PROCEDURE تعریف شده‌اند.

۶.۷ مدیریت خطا با TRY...CATCH

برای مدیریت خطاهای احتمالی داخل یک رویه (مثلاً نقض یک محدودیت FOREIGN KEY)، از بلوک TRY...CATCH استفاده می‌کنیم — مفهومی مشابه try/catch در بسیاری از زبان‌های برنامه‌نویسی:

```

1 CREATE PROCEDURE Sales.SafeAddPromotion
2     @PromotionName VARCHAR(50),
3     @ProductModelID INT,
4     @Discount DECIMAL(4,2)
5 AS
6 BEGIN
7     BEGIN TRY

```

```

8      INSERT INTO Sales.Promotion (PromotionName, ProductModelID,
9      Discount)
10     VALUES (@PromotionName, @ProductModelID, @Discount);
11
12     PRINT 'Promotion added successfully.';
13 END TRY
14 BEGIN CATCH
15     PRINT 'Error occurred: ' + ERROR_MESSAGE();
16     -- optionally log the error somewhere, or re-throw it:
17     -- THROW;
18 END CATCH
END;

```

توابع مفید داخل بلوک CATCH:

تابع توضیح	پ.۳۰ پ.۵۸
ERROR_MESSAGE()	متن کامل پیام خطا را برمی‌گرداند.
ERROR_NUMBER()	شماره‌ی خطای SQL Server را برمی‌گرداند.
ERROR_LINE()	شماره‌ی خطی که خطا در آن رخ داده را برمی‌گرداند.
ERROR_PROCEDURE()	نام رویه‌ای که خطا در آن رخ داده را برمی‌گرداند.

نکته‌ی مهم: دستور THROW (بدون آرگومان، داخل بلوک CATCH) همان خطای اصلی را دوباره به بیرون پرتاب می‌کند — این کار برای مواردی مفید است که می‌خواهید خطا را لاگ کنید اما همچنان اجازه دهید فراخواننده هم از وقوع خطا مطلع شود.

۷.۷ تغییر و حذف Procedure: ALTER PROCEDURE و DROP PROCEDURE

برای تغییر دادن یک رویه‌ی موجود (بدون نیاز به حذف و ساخت دوباره‌ی مجوزهای آن)، از ALTER PROCEDURE استفاده کنید — نحو آن دقیقاً مثل CREATE PROCEDURE است:

```

1 ALTER PROCEDURE Sales.GetRecentOrders
2     @DaysBack INT = 60    -- default changed from 30 to 60
3 AS
4 BEGIN
5     SELECT SalesOrderID, OrderDate, TotalDue
6     FROM Sales.SalesOrderHeader
7     WHERE OrderDate >= DATEADD(day, -@DaysBack, GETDATE());
8 END;

```

برای حذف کامل یک رویه:

```

1 DROP PROCEDURE Sales.GetRecentOrders;
2
3 -- safe version that does not error if the procedure does not exist:
4 DROP PROCEDURE IF EXISTS Sales.GetRecentOrders;

```

۸.۷ نکات و بهترین شیوه‌ها

- همیشه در ابتدای رویه، از SET NOCOUNT ON استفاده کنید تا پیام «تعداد ردیف‌های تحت تأثیر» بعد از هر دستور DML، به کلاینت فرستاده نشود — این کار حجم شبکه را کم می‌کند و کارایی را بهتر می‌کند.
- نام پارامترها را واضح و توصیفی انتخاب کنید (مثلاً @CustomerID به جای @id).

- از پیشوند اسکیما (مثل Sales) برای نام رویه استفاده کنید تا هم سازمان‌دهی بهتری داشته باشید، هم کارایی جست‌وجوی رویه توسط SQL Server کمی بهتر شود.

- برای رویه‌هایی که تغییرات چندمرحله‌ای در داده انجام می‌دهند (مثل چند INSERT و UPDATE پشت سر هم که باید همه یا هیچ‌کدام انجام شوند)، از BEGIN TRANSACTION، COMMIT و ROLLBACK در کنار TRY...CATCH استفاده کنید — این موضوع در حوزه‌ی Transaction Management است و می‌تواند موضوع یک فصل تکمیلی جداگانه باشد.

منبع

این فصل، بر خلاف شش فصل قبلی، خلاصه‌ی یک مازول واحد از مایکروسافت لرن نیست، بلکه تألیفی مستقل بر پایه‌ی مستندات عمومی T-SQL است. برای مطالعه‌ی رسمی و تکمیلی، می‌توانید به این منبع مراجعه کنید:

[CREATE PROCEDURE \(Transact-SQL\) — Microsoft Learn](#)